PROJETO ACADÊMICO PJBL — ATIVIDADE EM GRUPO

Documento de Arquitetura de Software

Integrador de Orçamentos

Cliente Schulz S/A — desenvolvido por SKA Automação de Engenharias Ltda

arc42 (11 seções) • C4 Model (níveis 1 a 4) • UML 2.5 (componentes, classes, sequência e implantação) • Diagrama de Entidades e Relacionamentos • Software Architecture Canvas

Integrantes do grupo:
Alexandre Filho • Ana Julia • Felipe Nathan • Matheus Silva

Repositório no GitHub	https://github.com/ttheus27/Projeto_Claud
Aplicação em produção (GitHub Pages)	https://ttheus27.github.io/Projeto_Claud/
API serverless (Azure Function App)	https://cloud-projeto-orc-f-app-aah7f0c3ezdweygp.canadacentral-01.azurewebsites.net/api
Banco de dados	MongoDB Atlas — banco schulz (acesso restrito)
Versão do documento	2.0 — outubro de 2026

Documento estruturado conforme o template oficial arc42 (11 seções e glossário).

Sumário

Sumário executivo
1. Introdução e Objetivos (Introduction and Goals)
    1.1 Visão geral dos requisitos (Requirements Overview)
    1.2 Metas de qualidade (Quality Goals)
    1.3 Partes interessadas (Stakeholders)
2. Restrições da Arquitetura (Architecture Constraints)
    2.1 Restrições técnicas
    2.2 Restrições organizacionais
    2.3 Convenções adotadas
3. Escopo e Contexto do Sistema (Context and Scope)
    3.1 Contexto de negócio (Business Context)
    3.2 Contexto técnico (Technical Context)
4. Estratégia de Solução (Solution Strategy)
    4.1 Software Architecture Canvas
    4.2 Estilos arquiteturais considerados
5. Visão de Blocos de Construção (Building Block View)
    5.1 Nível 1 de decomposição — contêineres (C4 Model nível 2)
    5.2 Nível 2 de decomposição — componentes do Frontend SPA (C4 nível 3)
    5.3 Nível 2 de decomposição — componentes da API Serverless (C4 nível 3)
    5.4 Nível 3 de decomposição — código (C4 nível 4)
6. Visão de Runtime (Runtime View)
    6.1 Consulta de orçamentos e indicadores
    6.2 Flag Zerar Custo e recálculo instantâneo
    6.3 Gravação do orçamento recalculado
    6.4 Proposta comercial e exportação estruturada
7. Visão de Implantação (Deployment View)
    7.1 Configuração por ambiente
    7.2 Procedimento de publicação
8. Conceitos Transversais (Crosscutting Concepts)
    8.1 Modelo de domínio e de dados
    8.2 Regra de precificação
    8.3 Contrato HTTP, erros e CORS
    8.4 Configuração, segredos e ambientes
    8.5 Segurança e transporte
    8.6 Observabilidade
    8.7 Estratégia de testes
    8.8 Experiência de uso e desempenho percebido
9. Decisões de Arquitetura (Architecture Decisions)

9.1 Decisões estruturantes
9.2 Estilos arquiteturais considerados e não adotados
9.3 Garantia estrutural e publicação
10. Requisitos de Qualidade (Quality Requirements)
10.1 Árvore de qualidade
10.2 Cenários de qualidade
11. Riscos e Dívidas Técnicas (Risks and Technical Debt)
12. Glossário (Glossary)

Lista de figuras

Figura 1 — C4 Model — Nível 1: diagrama de contexto do Integrador de Orçamentos

Figura 2 — Contexto técnico: canais, protocolos e plataformas que cercam o sistema

Figura 3 — Software Architecture Canvas do Integrador de Orçamentos

Figura 4 — Mapa dos estilos arquiteturais: aplicados no produto e deliberadamente não adotados, com a ADR correspondente

Figura 5 — C4 Model — Nível 2: diagrama de contêineres

Figura 6 — C4 Model — Nível 3: componentes do contêiner Frontend SPA

Figura 7 — C4 Model — Nível 3: componentes do contêiner API Serverless

Figura 8 — C4 Model — Nível 4: diagrama de componentes UML com interfaces fornecidas e requeridas

Figura 9 — C4 Model — Nível 4: classes do modelo de domínio e do motor de cálculo de custos

Figura 10 — C4 Model — Nível 4: classes de casos de uso, porta e adaptadores do frontend

Figura 11 — C4 Model — Nível 4: classes da API serverless de orçamentos

Figura 12 — Sequência UML: consulta de orçamentos e composição dos indicadores (RF02)

Figura 13 — Sequência UML: flag Zerar Custo e recálculo imediato do preço (RF03)

Figura 14 — Sequência UML: gravação do orçamento com validação de domínio (RF06)

Figura 15 — Sequência UML: emissão da proposta comercial e exportação em JSON (RF05, RF10)

Figura 16 — Diagrama de implantação UML: nós de execução, artefatos e canais

Figura 17 — Diagrama de Entidades e Relacionamentos do banco de documentos schulz

Figura 18 — Regras de dependência verificadas pelos testes unitários de arquitetura

Sumário executivo

Este documento descreve a arquitetura do **Integrador de Orçamentos**, sistema web que apoia a orçamentação de peças usinadas e compressores da **Schulz S/A**, desenvolvido pela **SKA Automação de Engenharias Ltda**. A organização do texto segue o **template oficial arc42 em suas 11 seções**, complementado pelo **glossário** previsto no template e enriquecido com as notações pedidas na disciplina: **C4 Model nos níveis 1, 2, 3 e 4**, **UML 2.5** (componentes, classes, sequência e implantação), **Diagrama de Entidades e Relacionamentos** e **Software Architecture Canvas**.

A solução descrita aqui é a que está implementada no repositório do grupo. Ela combina quatro decisões estruturantes: uma **SPA React** que executa o motor de cálculo de custos no próprio navegador, uma **API serverless em Azure Functions** que concentra a persistência, um **banco de documentos no MongoDB Atlas** que guarda o orçamento como agregado único e uma organização interna de código em **Clean Architecture com fatias verticais (Vertical Slice)**, cujas fronteiras são verificadas por **testes unitários de arquitetura**.

O destaque funcional é a regra que motivou o projeto: a **flag Zerar Custo** por operação. Operações bonificadas, retrabalhos internos e inspeções metrológicas não devem ser repassadas ao preço da peça, mas seus tempos continuam valendo para o planejamento de produção. Como a regra de precificação é uma função pura executada no cliente, o orçamentista vê o preço recalculado na mesma interação em que aciona a flag, sem esperar o servidor.

Aspecto	Decisão desta arquitetura
Estilo de interface	SPA React 18 com renderização no cliente, navegação por abas no shell e publicação como artefato estático
Estilo de backend	Serverless: quatro funções HTTP em Azure Functions v4, uma por operação de orçamento
Organização do código	Clean Architecture com Ports & Adapters e fatias verticais no frontend e na API
Persistência	Banco de documentos (MongoDB Atlas) com o orçamento e suas operações no mesmo agregado
Local da regra de negócio	Função pura de domínio no cliente, recalculada a cada interação e nunca persistida
Garantia estrutural	Doze testes unitários de arquitetura que falham se a regra de dependência for violada

1. Introdução e Objetivos (Introduction and Goals)

A Schulz S/A é uma das maiores fabricantes de compressores de ar da América Latina e fornecedora de peças fundidas e usinadas para os setores automotivo e industrial. Seu portfólio inclui virabrequins forjados, suportes de pinça de freio em ferro fundido nodular, carcaças de compressores de parafuso e pistões de alta precisão. Cada proposta comercial depende de um levantamento técnico detalhado: material, dureza, peso, lote de fabricação, sequência de operações CNC, tempos de setup e de ciclo, custo de ferramental, centro de custo e tolerâncias de desenho.

Historicamente esse levantamento era feito em planilhas locais e em um sistema legado desktop mantido pela SKA, o que espalhava a informação, dificultava a padronização da folha de processo e tornava cada revisão de preço um retrabalho manual. O Integrador de Orçamentos unifica esse fluxo em uma aplicação web: centraliza os dados da peça e das operações, calcula o custo de cada operação, aplica margem e impostos, apresenta o desenho técnico e emite a proposta formal do módulo Integrador_Reports.

1.1 Visão geral dos requisitos (Requirements Overview)

O escopo funcional foi organizado em dez requisitos macro, acordados com a engenharia de manufatura e com a área comercial. A coluna de critério de aceite descreve o comportamento esperado do sistema para cada requisito.

Código	Requisito funcional	Prior.	Ator	Critério de aceite
RF01	Importar dados de orçamento a partir das planilhas e bases Schulz (.xlsx, .ods)	Alta	Usuário de Orçamentos	Linhas da planilha convertidas em orçamento estruturado, com pré-visualização antes da gravação
RF02	Consultar orçamentos cadastrados com filtros por cliente, peça e situação	Alta	Usuário de Orçamentos / Gestor	Busca por código, descrição, cliente ou identificador, com indicadores consolidados na mesma tela
RF03	Calcular custos de usinagem por operação, com possibilidade de zerar o custo por flag	Alta	Usuário de Orçamentos	Recálculo imediato de setup amortizado, ciclo, homem-hora, ferramental, margem e impostos ao acionar a flag
RF04	Armazenar e exibir o desenho técnico vinculado ao item orçado	Média	Usuário de Orçamentos	Visualização vetorial com cotas, tolerância geral ISO 2768-mK e rugosidade Ra
RF05	Gerar a proposta comercial consolidada do módulo Integrador_Reports	Alta	Usuário / Gestor Comercial	Documento paginado em A4 com identificação Schulz/SKA, folha de processo e demonstrativo de preço
RF06	Editar peças, operações, centros de custo, margem e lote	Média	Usuário de Orçamentos	Inclusão, alteração e remoção de operações com gravação no banco e confirmação na tela
RF07	Validar a integridade dos dados importados antes da gravação	Média	Usuário de Orçamentos	Verificação de consistência apresentada ao usuário, que confirma ou cancela a importação
RF08	Controlar o acesso conforme o perfil (operação, gestão e TI)	Média	Administrador de TI	Perfis distintos para edição de taxas e aprovação de margens

Código	Requisito funcional	Prior.	Ator	Critério de aceite
RF09	Registrar o histórico de alterações dos orçamentos	Baixa	Administrador de TI	Marcação temporal de criação e atualização em cada orçamento e registro de invocação na telemetria
RF10	Exportar o orçamento em formato estruturado para outros sistemas	Média	Gestor / Integração	Arquivo JSON com metadados da peça, operações e composição completa do preço

1.2 Metas de qualidade (Quality Goals)

As cinco metas a seguir, alinhadas aos atributos da ISO/IEC 25010, orientaram as decisões desta arquitetura. Elas aparecem detalhadas em cenários mensuráveis na seção 10.

Prior.	Atributo	Meta de arquitetura	Motivação
1	Reatividade (eficiência de desempenho)	O preço completo é recalculado na mesma interação do usuário, sem chamada de rede	O orçamentista compara cenários alterando tempos, lote e flags; esperar o servidor interromperia o raciocínio
2	Testabilidade (manutenibilidade)	A regra de precificação é executável sem navegador, sem rede e sem banco de dados	Preço errado gera prejuízo direto; a regra precisa de teste automatizado rápido e determinístico
3	Modificabilidade (manutenibilidade)	Mudança de funcionalidade fica contida em uma fatia vertical, e as fronteiras são verificadas automaticamente	Equipe pequena, projeto em evolução: a estrutura precisa resistir à erosão sem depender de revisão manual
4	Elasticidade e custo operacional	A API escala por invocação e não há servidor ocioso para administrar	O uso é concentrado em horário comercial e o orçamento de infraestrutura é restrito
5	Portabilidade de publicação	O frontend é um artefato estático com caminhos relativos, publicável em mais de um provedor	A hospedagem acadêmica usa GitHub Pages, mas a solução precisa caber em Azure Static Web Apps sem reescrita

1.3 Partes interessadas (Stakeholders)

Parte interessada	Papel	Expectativa em relação à arquitetura
Usuário de Orçamentos	Elabora e revisa orçamentos de peças usinadas	Resposta imediata ao alterar operações e flags; confiança no cálculo apresentado
Gestor Comercial	Acompanha indicadores e emite propostas	Indicadores consolidados, proposta formal pronta para envio e exportação de dados
Administrador de TI / SKA	Publica e mantém a solução	Publicação simples, configuração por variáveis de ambiente e telemetria de falhas
Equipe de desenvolvimento	Evolui o produto	Fronteiras explícitas entre camadas, testes rápidos e código previsível
Cliente industrial	Recebe a proposta comercial	Documento padronizado, com composição de custos rastreável
Banca avaliadora	Avalia a entrega acadêmica	Documentação completa e coerente com o código entregue

2. Restrições da Arquitetura (Architecture Constraints)

As restrições a seguir delimitaram o espaço de solução. Elas não são preferências do grupo: são condições de contorno impostas pelo contexto acadêmico, pela plataforma escolhida e pelo próprio domínio industrial.

2.1 Restrições técnicas

Restrição	Consequência para a arquitetura
Publicação do frontend em hospedagem estática (GitHub Pages)	Não há servidor de aplicação para o frontend: a SPA precisa funcionar com caminhos relativos (base `./`) e sem rota de fallback no servidor, o que levou à navegação por estado em vez de rotas de URL
Camadas gratuitas de Azure e MongoDB Atlas	Plano de consumo nas funções, com partida a frio aceitável, e reaproveitamento da conexão de banco entre invocações do mesmo worker
JavaScript em frontend e backend, sem etapa de compilação de tipos	Os contratos entre camadas são verificados em tempo de execução e por testes, em vez de interfaces de tipagem estática
Comunicação exclusivamente HTTP/JSON entre SPA e API	O frontend nunca abre conexão direta com o banco; toda leitura e gravação atravessa as funções
Navegadores corporativos atualizados	É possível usar Fetch API, módulos ES e recursos modernos de CSS sem camada de compatibilidade
Segredos fora do repositório	A string de conexão e demais parâmetros vivem em variáveis de ambiente do Function App e em arquivos locais não versionados

2.2 Restrições organizacionais

Restrição	Consequência para a arquitetura
Entrega acadêmica em grupo de quatro integrantes, com prazo fixo	Escopo recortado em uma fatia vertical de orçamentos, profunda em arquitetura e enxuta em superfície funcional
Repositório único para frontend, API e documentação	Dois pacotes independentes (`Projeto/` e `functions/`) convivendo no mesmo versionamento, com publicação separada
Publicação da API feita manualmente pelo administrador	A esteira automatizada cobre o frontend; a função é publicada com as ferramentas oficiais da Azure
Documentação de arquitetura como artefato versionado	Diagramas mantidos como código-fonte PlantUML e reconstruídos por script, evitando imagens desatualizadas

2.3 Convenções adotadas

Convenção	Detalhe
Notações de documentação	arc42 para a estrutura, C4 Model para a visão estática em quatro níveis, UML 2.5 para componentes, classes, sequência e implantação, DER para o modelo de dados e Software Architecture Canvas para a visão de uma página
Idioma	Português do Brasil no documento, nos nomes de domínio e nas mensagens ao usuário; termos técnicos consagrados mantidos no original

Convenção	Detalhe
Organização de pastas	Fatia vertical por funcionalidade e, dentro dela, camadas `domain`, `application`, `infrastructure`, `presentation` e `composition`
Nomenclatura de testes	Cenários escritos em DADO/QUANDO/ENTÃO, em linguagem de domínio e não de implementação
Geração dos diagramas	PlantUML com C4-PlantUML, renderização em 200 dpi por script Node.js, para que texto e proporção permaneçam legíveis no PDF

3. Escopo e Contexto do Sistema (Context and Scope)

Esta seção delimita o que pertence ao sistema e quem está do lado de fora. O contexto de negócio apresenta os atores e os fluxos de informação; o contexto técnico mostra os canais e protocolos por onde esses fluxos passam.

3.1 Contexto de negócio (Business Context)

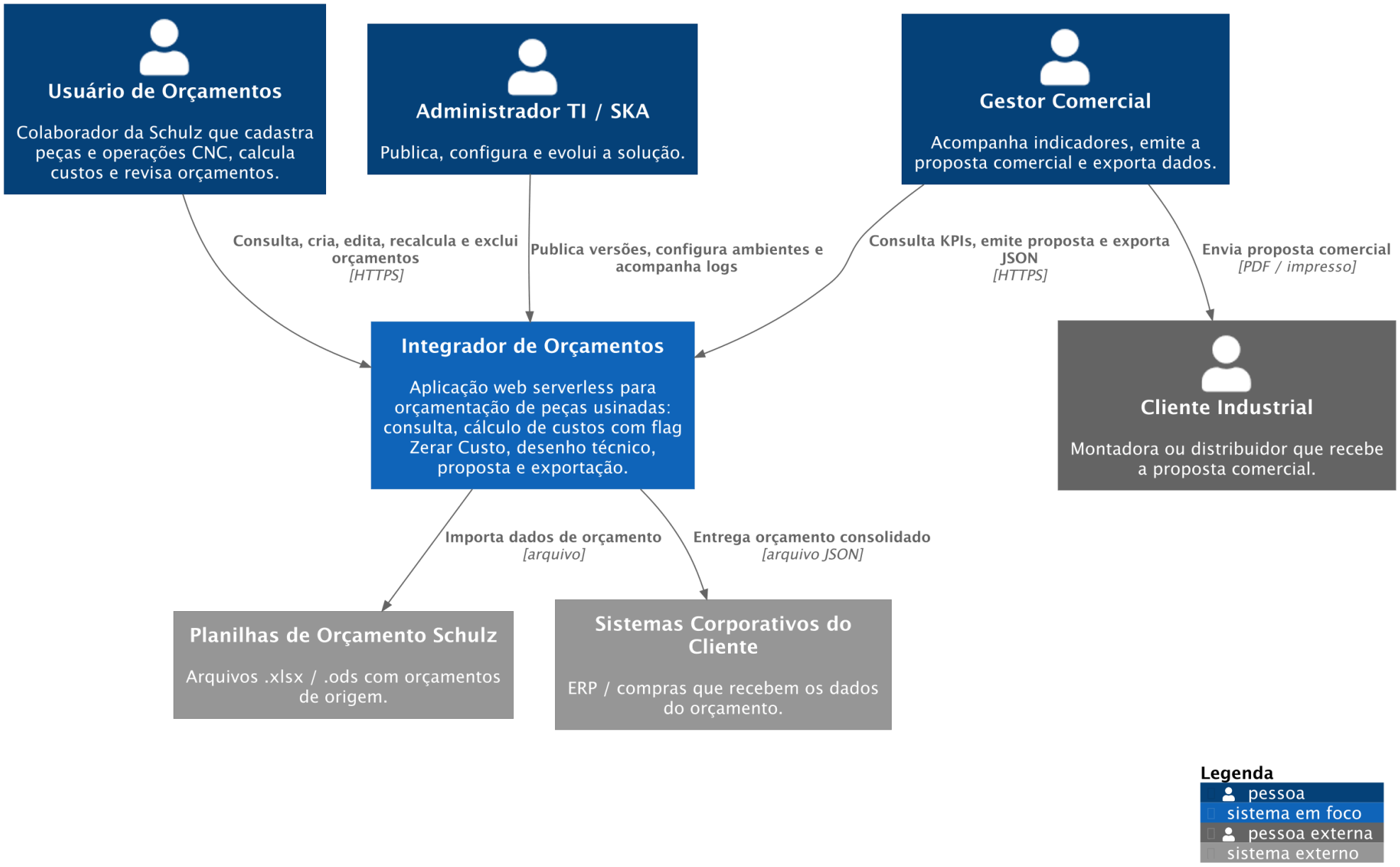


Figura 1 — C4 Model — Nível 1: diagrama de contexto do Integrador de Orçamentos

Ator ou sistema vizinho	Entrada que fornece	Saída que recebe
Usuário de Orçamentos	Dados da peça, folha de processo, tempos, custos e flags de operação	Preço unitário e do lote recalculados, desenho técnico e confirmação de gravação
Gestor Comercial	Critérios de filtro, margem e decisão de aprovação	Indicadores consolidados, proposta comercial paginada e exportação estruturada
Administrador de TI / SKA	Versões publicadas, configuração de ambiente e carga de dados de exemplo	Telemetria de invocações e erros da API
Planilhas de orçamento Schulz	Arquivos .xlsx e .ods com orçamentos de origem	—
Sistemas corporativos do cliente (ERP / compras)	—	Arquivo JSON com a árvore completa do orçamento
Cliente industrial	—	Proposta comercial impressa ou em PDF

3.2 Contexto técnico (Technical Context)

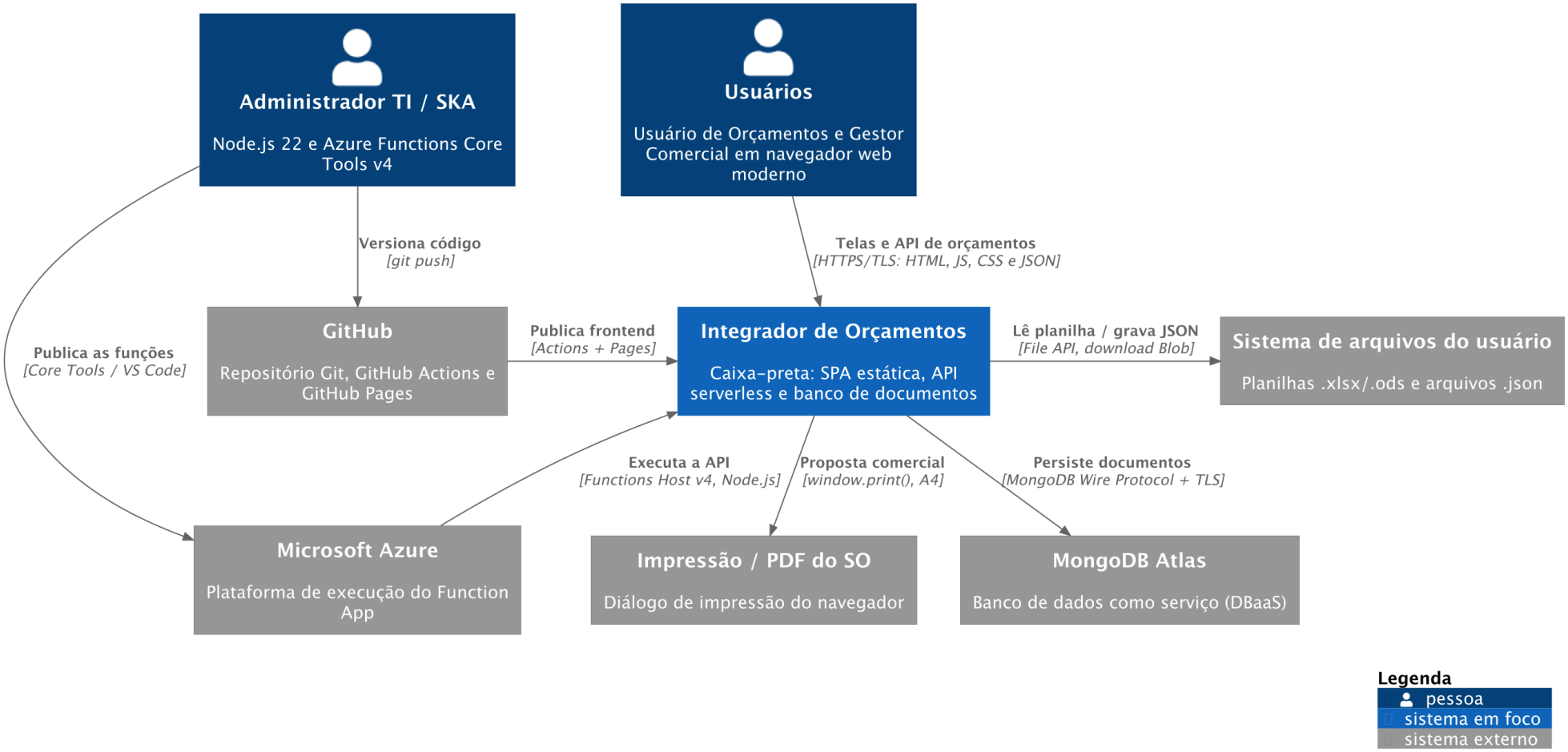


Figura 2 — Contexto técnico: canais, protocolos e plataformas que cercam o sistema

Canal	Protocolo e formato	Uso
Navegador ↔ GitHub Pages	HTTPS/TLS, HTML, JavaScript e CSS estáticos	Entrega da SPA e de seus recursos
Navegador ↔ Azure Function App	HTTPS/JSON, métodos GET, POST, PUT, DELETE e OPTIONS	Consulta e gravação de orçamentos
Function App ↔ MongoDB Atlas	MongoDB Wire Protocol sobre TLS, Stable API v1	Persistência do agregado de orçamento
Navegador ↔ sistema de arquivos	File API para leitura e Blob para download	Importação de planilha e exportação do JSON
Navegador ↔ serviço de impressão	Diálogo de impressão do sistema operacional, página A4	Emissão da proposta comercial em PDF ou papel
GitHub Actions ↔ GitHub Pages	Artefato de build publicado pela própria esteira	Publicação automatizada do frontend
Estação do administrador ↔ Azure	Azure Functions Core Tools v4	Publicação e execução local das funções

4. Estratégia de Solução (Solution Strategy)

A estratégia nasce do cruzamento entre as metas de qualidade da seção 1.2 e as restrições da seção 2. Cada meta foi endereçada por uma decisão estrutural, registrada em detalhe como ADR na seção 9.

Meta de qualidade	Abordagem adotada	Onde isso aparece
Reatividade do recálculo	Motor de custos como função pura executada no navegador, sem ida ao servidor para recalcular	`domain/calcular-custos-orcamento.js` e a sequência da figura 13
Testabilidade da regra	Domínio sem dependências e casos de uso recebendo portas por injeção, substituíveis por dublês	Casos de uso com gateway em memória nos testes
Modificabilidade	Clean Architecture com Ports & Adapters dentro de fatias verticais, uma por funcionalidade	Fatia `features/orcamentos` e fatias `create`, `list`, `update` e `delete`
Proteção das fronteiras	Testes unitários de arquitetura que leem o código e falham quando a regra de dependência é quebrada	`src/test/arquitetura/regras-de-dependencia.test.js`
Elasticidade e custo	API serverless com uma função HTTP por operação e conexão de banco reaproveitada entre invocações	Azure Functions v4 e `shared/infra/mongodb/client.js`
Simplicidade de leitura dos dados	Orçamento e operações no mesmo documento, dispensando junção para montar a tela	Coleção `orcamentos` no MongoDB Atlas
Portabilidade de publicação	Build estático com caminhos relativos e configuração de endpoints por variável de ambiente	`vite.config.js` com `base: "./"` e variáveis `VITE_*`

4.1 Software Architecture Canvas

O canvas a seguir resume em uma página o problema, as partes interessadas, o escopo, as decisões estruturantes, os atributos de qualidade, as restrições, os blocos de construção, os dados, as tecnologias, a verificação de qualidade, a operação e as decisões tomadas sobre os estilos arquiteturais de referência.

Software Architecture Canvas — Integrador de Orçamentos

Schulz S/A (cliente industrial) • SKA Automação de Engenharias Ltda (fornecedor) • Projeto acadêmico PJBL

Versão 2.0 — outubro de 2026
Alexandre Filho • Ana Julia • Felipe Nathan • Matheus Silva

1. PROBLEMA E OBJETIVO DE NEGÓCIO

- Orçamentação de peças usinadas e compressores dispersa em planilhas locais e em sistema legado desktop.
- Reduzir tempo e retrabalho na formação de preço de peças usinadas, padronizando a folha de processo CNC.
- Acelerar a emissão de propostas formais para montadoras e distribuidores.
- Centralizar peça, operações, custos e desenho técnico.

2. PARTES INTERESSADAS

- **Usuário de Orçamentos:** cadastra peças e operações, aciona a flag de custo e revisa preços.
- **Gestor Comercial:** acompanha indicadores, emite a proposta e exporta dados.
- **Administrador de TI / SKA:** publica versões, configura ambientes e acompanha a telemetria.
- **Cliente industrial:** recebe a proposta comercial.

3. ESCOPO FUNCIONAL

- Importação de orçamentos de planilhas Schulz (RF01, RF07).
- Consulta com filtros e indicadores (RF02).
- Cálculo de custos por operação com flag **Zerar Custo** (RF03).
- Desenho técnico com cotas e tolerâncias (RF04).
- Proposta formal Integrador_Reports (RF05).
- Edição de peças, operações e centros de custo (RF06).
- Exportação estruturada em JSON (RF10).

4. DECISÕES ARQUITETURAIS ESTRUTURANTES

- **Serverless:** API em Azure Functions v4, custo por invocação.
- **Clean Architecture:** domínio puro no centro; dependências apontando para dentro.
- **Vertical Slice:** fatia *features/orcamentos* no frontend; fatias create, list, update e delete na API.
- **Ports & Adapters:** portas OrcamentoGateway e OrcamentoRepository com injeção no composition root.
- **Agregado único:** orçamento e operações no mesmo documento.

5. ATRIBUTOS DE QUALIDADE PRIORIZADOS

- **Reatividade:** recálculo de preço na própria interação, sem ida ao servidor.
- **Testabilidade:** regra de negócio pura, testável sem navegador, rede ou banco.
- **Manutenibilidade:** fronteiras verificadas por testes de arquitetura.
- **Elasticidade e custo:** escala por invocação, sem servidor ocioso.
- **Portabilidade:** artefato estático publicável em Pages ou Static Web Apps.

6. RESTRIÇÕES

- Entrega acadêmica em grupo de quatro integrantes, com prazo fixo.
- Publicação do frontend limitada a hospedagem estática (GitHub Pages).
- Plataforma de nuvem e banco restritos às camadas gratuitas de Azure e MongoDB Atlas.
- JavaScript em frontend e backend, sem etapa de compilação de tipos.
- Segredos fora do repositório: configuração por variáveis de ambiente.

7. BLOCOS DE CONSTRUÇÃO

- **Frontend SPA:** React 18, Vite 6 e Tailwind; navegação por abas no shell.
- **API serverless:** quatro funções HTTP com caso de uso, domínio, repositório e presenter.
- **Banco de documentos:** MongoDB Atlas, banco *schulz*.
- **Script de carga:** seed idempotente por *upsert*.
- **Esteira de publicação:** GitHub Actions para o frontend.

8. DADOS E INTEGRAÇÕES

- Agregado **orcamentos**: metadados da peça, operações embutidas e desenho técnico; índice único por código de negócio.
- Catálogo **centros_de_custo** com taxa horária média.
- Entrada: planilhas .xlsx e .ods do cliente.
- Saída: proposta em PDF e payload JSON para ERP e compras.
- Custos calculados nunca são persistidos: são derivados.

9. TECNOLOGIAS E PLATAFORMA

- React 18, Vite 6, Tailwind CSS 3, Lucide React.
- Azure Functions v4 (Node.js), driver oficial MongoDB 7.
- MongoDB Atlas em cluster gerenciado.
- GitHub Actions e GitHub Pages.
- Vitest, Testing Library e jsdom.
- PlantUML e C4-PlantUML para os diagramas.

10. QUALIDADE VERIFICADA

- Testes de domínio cobrindo composição de preço e flag de custo zerado.
- Testes de caso de uso com gateway em memória e relógio controlado.
- Teste de interface do filtro do dashboard.
- **Testes unitários de arquitetura:** doze regras de dependência no frontend e na API.
- Nomenclatura GIVEN/WHEN/THEN em linguagem de domínio.

11. OPERAÇÃO E SEGURANÇA

- Configuração por variáveis de ambiente no Function App e em arquivos não versionados.
- Transporte sempre em HTTPS/TLS, inclusive até o Atlas.
- Telemetria de invocações e erros no Application Insights.
- Conexão de banco reaproveitada entre invocações do mesmo worker.
- Recuperação do ambiente por nova publicação e recarga do seed.

12. ESTILOS DELIBERADAMENTE NÃO ADOTADOS

- **Microfrontend:** uma equipe e um domínio central; artefato de publicação único (ADR-05).
- **BFF + microsserviços + banco por serviço:** um consumidor e um agregado transacional; serviço coeso (ADR-06).
- **API Gateway:** prefixo `/api` já é o contrato de borda estável (ADR-07).
- **EDA:** fluxo interativo e sincrônico, com consistência imediata em tela (ADR-08).

Integrador de Orçamentos — documentação de arquitetura arc42, C4 Model e UML

Software Architecture Canvas

Figura 3 — Software Architecture Canvas do Integrador de Orçamentos

4.2 Estilos arquiteturais considerados

A disciplina apresentou um conjunto de estilos arquiteturais de referência. Todos foram considerados nesta arquitetura e, para cada um, existe uma decisão registrada: parte está **aplicada** na estrutura do produto e parte foi **deliberadamente não adotada** neste escopo, com o critério que sustenta a escolha documentado nas ADRs da seção 9. O mapa abaixo mostra essa distribuição.

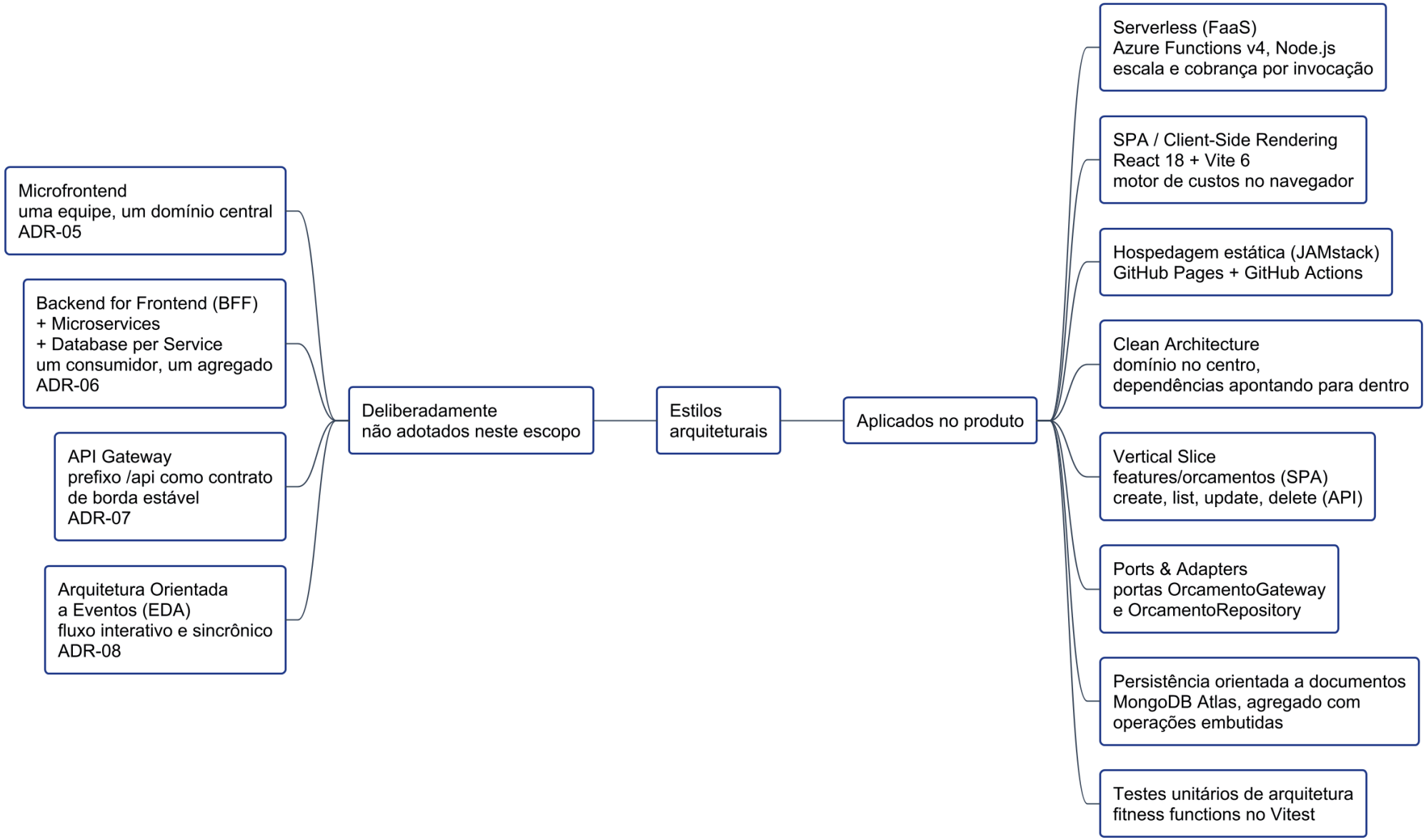


Figura 4 — Mapa dos estilos arquiteturais: aplicados no produto e deliberadamente não adotados, com a ADR correspondente

Estilo	Situação nesta arquitetura	Referência
Serverless	Aplicado: API em Azure Functions v4, uma função HTTP por operação, com escala e cobrança por invocação	ADR-01
Clean Architecture	Aplicado: domínio puro no centro, aplicação orquestrando portas e adaptadores na borda, nas duas pontas do sistema	ADR-02
Vertical Slice	Aplicado: fatia `features/orcamentos` no frontend e fatias `create`, `list`, `update` e `delete` na API	ADR-03
Persistência orientada a documentos	Aplicado: orçamento como agregado único no MongoDB Atlas, com operações embutidas	ADR-04
Microfrontend	Não adotado: com uma equipe e um domínio central, a decisão foi publicar um artefato único, mantendo a fatia vertical como unidade de isolamento	ADR-05
Backend for Frontend (BFF) + Microservices + Database per Service	Não adotado: com um único consumidor e um único agregado transacional, a decisão foi manter um serviço coeso de orçamentação	ADR-06
API Gateway	Não adotado: o Function App já concentra roteamento e políticas de CORS sob um prefixo estável, o que dispensa uma camada dedicada	ADR-07
Arquitetura Orientada a Eventos (EDA)	Não adotado: o fluxo de orçamentação é interativo e sincrônico, e a tela depende da resposta para prosseguir	ADR-08
Testes unitários de arquitetura	Aplicado: doze regras de dependência verificadas a cada execução da suíte	ADR-09
Hospedagem estática (JAMstack)	Aplicado: artefato estático publicado por esteira automatizada, com caminhos relativos	ADR-10

5. Visão de Blocos de Construção (Building Block View)

Esta seção apresenta a decomposição estática do sistema, do todo para a parte, usando o **C4 Model**. O nível 1 (contexto) está na seção 3.1; aqui seguem o nível 2 (contêineres), o nível 3 (componentes de cada contêiner) e o nível 4 (código), este último expresso em diagramas UML de componentes e de classes.

5.1 Nível 1 de decomposição — contêineres (C4 Model nível 2)

O sistema é composto por quatro contêineres: a aplicação de página única executada no navegador, a API serverless, o script de carga de dados e o banco de documentos. O navegador conversa apenas com a API; apenas a API conversa com o banco.

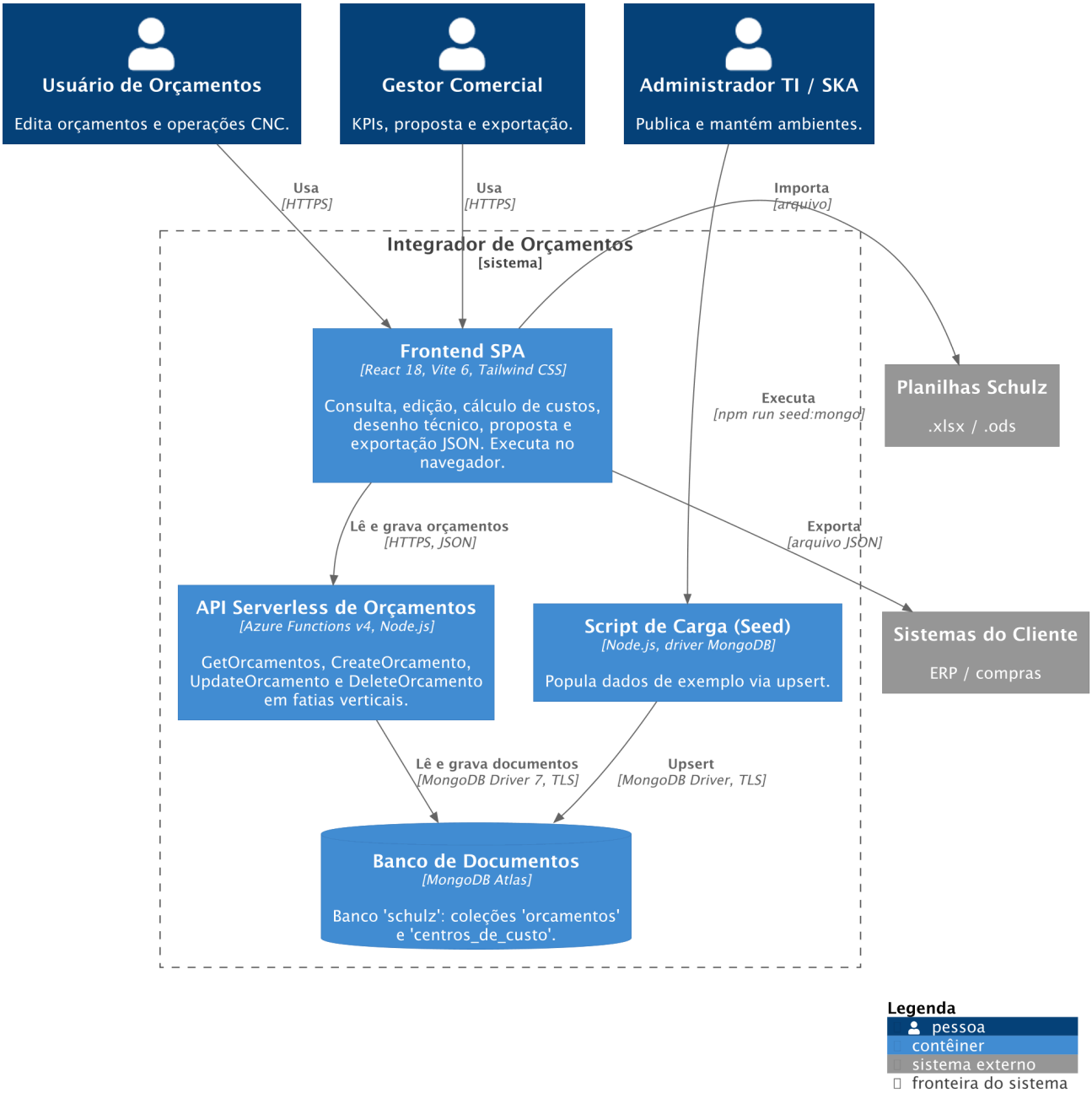


Figura 5 — C4 Model — Nível 2: diagrama de contêineres

Contêiner	Tecnologia	Responsabilidade
Frontend SPA	React 18, Vite 6, Tailwind CSS, Lucide	Apresentar o dashboard, o formulário de orçamento, o desenho técnico e a proposta; executar o motor de cálculo de custos; orquestrar os casos de uso
API Serverless de Orçamentos	Azure Functions v4, Node.js	Expor as operações de consulta, criação, atualização e remoção; validar o formato do orçamento; traduzir domínio e erros em respostas HTTP
Script de carga (seed)	Node.js com driver MongoDB	Criar coleções e índices e carregar orçamentos e centros de custo de exemplo de forma idempotente
Banco de documentos	MongoDB Atlas (banco `schulz`)	Guardar o agregado de orçamento com suas operações embutidas e o catálogo de centros de custo

5.2 Nível 2 de decomposição — componentes do Frontend SPA (C4 nível 3)

Dentro da SPA existe uma fatia vertical, `features/orcamentos`, organizada nas camadas da Clean Architecture. O shell `App` concentra o estado de tela e aciona os casos de uso; a camada de apresentação recebe dados e callbacks por propriedades; o domínio não conhece React; a infraestrutura é o único lugar que sabe que existe HTTP.

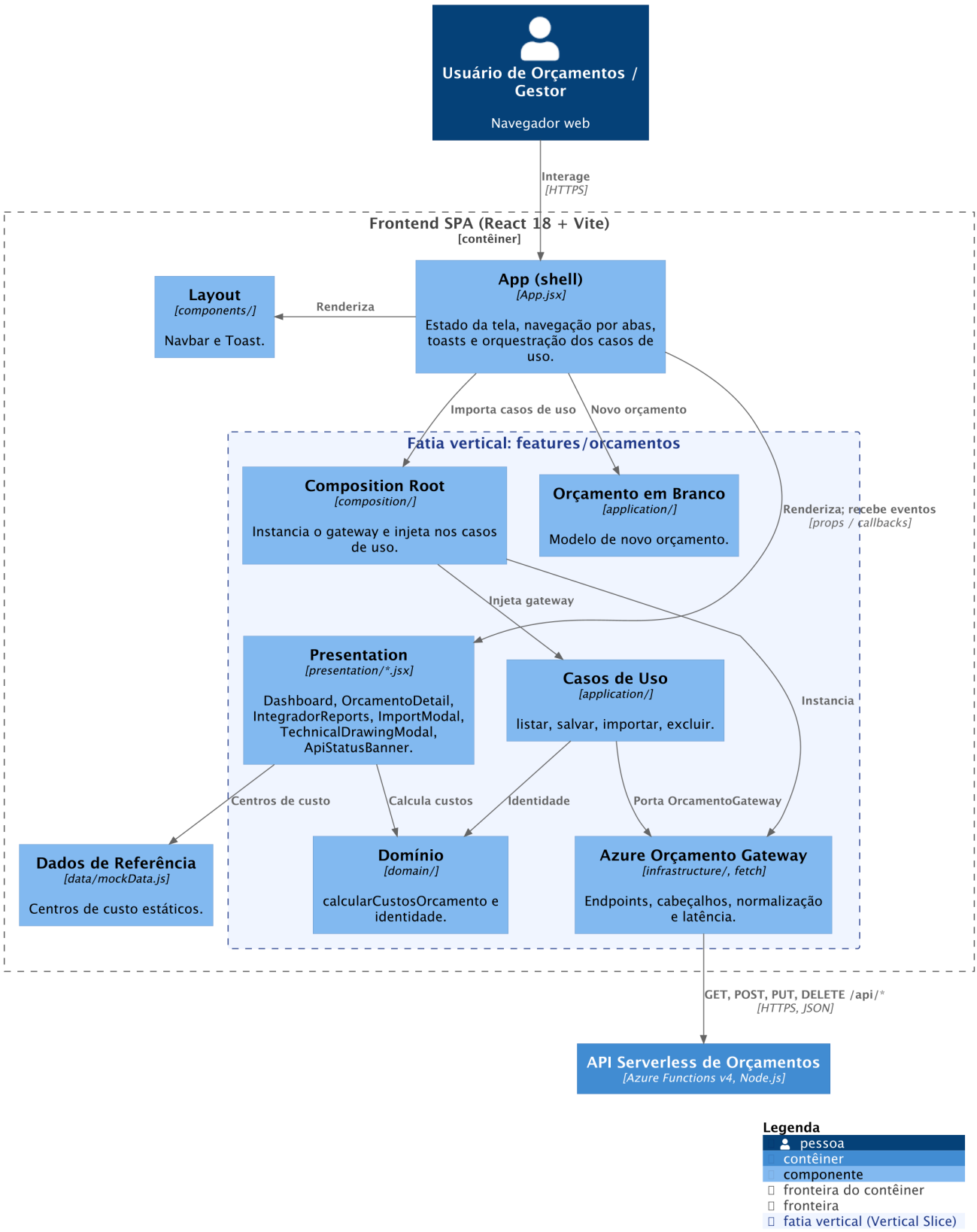


Figura 6 — C4 Model — Nível 3: componentes do contêiner Frontend SPA

Componente	Camada	Responsabilidade
`App`	shell de apresentação	Estado de navegação e de dados da tela, orquestração dos casos de uso, avisos ao usuário e recarga da base após cada gravação
`Dashboard`, `OrcamentoDetail`, `IntegradorReports`, `TechnicalDrawingModal`, `ImportModal`, `ApiStatusBanner`	presentation	Telas e componentes da fatia: consulta com filtros e indicadores, edição da folha de processo, proposta comercial, desenho técnico, importação e situação da API
`orcamentoUseCases`	application	Casos de uso listar, salvar, importar e excluir, mais a validação do contrato da porta
`criarOrcamentoEmBranco`	application	Monta o modelo inicial de um novo orçamento, com lote, margem, impostos e primeira operação
`calcularCustosOrcamento`, `getOrcamentoId`, `isOrcamentoPersistido`	domain	Regra de precificação e identidade do orçamento, sem qualquer dependência externa
`createAzureOrcamentoGateway`	infrastructure	Adaptador HTTP: endpoints, cabeçalhos, chave de função, normalização de resposta e medição de latência
`orcamentoUseCases` / `orcamentoApiConfiguration`	composition	Composition root da fatia: instancia o adaptador, injeta na aplicação e publica a configuração para a apresentação
`Navbar`, `Toast`	componentes compartilhados	Navegação entre abas e mensagens de retorno, sem regra de negócio
`data/mockData`	dados de referência	Catálogo de centros de custo e respectiva taxa horária média usados nos formulários

5.3 Nível 2 de decomposição — componentes da API Serverless (C4 nível 3)

A API aplica a mesma ideia em outra granularidade: cada operação é uma fatia (`create`, `list`, `update`, `delete`) formada pelo adaptador HTTP e pelo seu caso de uso. As fatias compartilham o domínio, o repositório, o apresentador e a infraestrutura de resposta e de conexão. O arquivo de inicialização apenas registra as quatro funções.

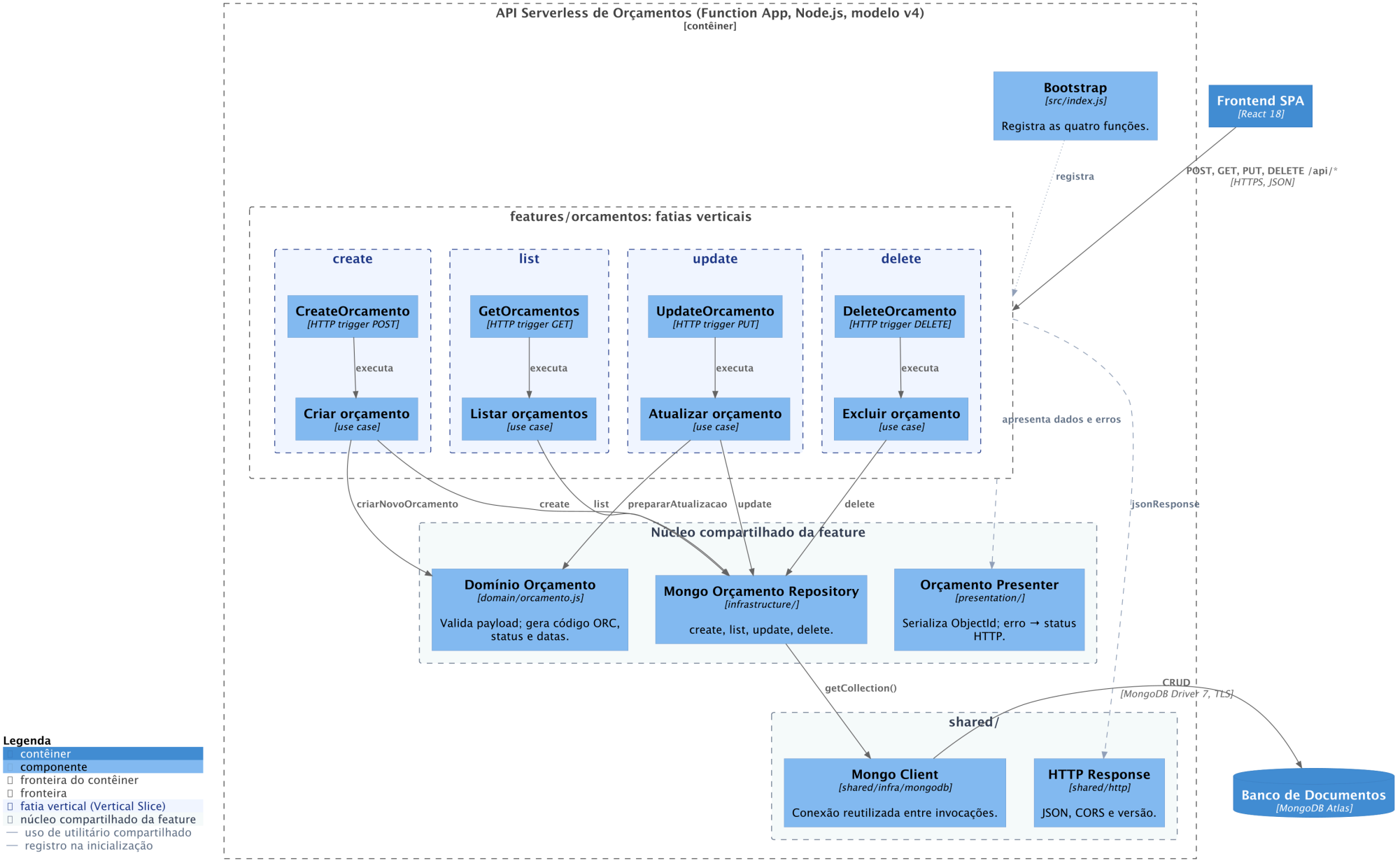


Figura 7 — C4 Model — Nível 3: componentes do contêiner API Serverless

Componente	Papel	Responsabilidade
`src/index.js`	inicialização	Habilita o modelo v4 e registra as quatro funções HTTP por importação
`GetOrcamentos`, `CreateOrcamento`, `UpdateOrcamento`, `DeleteOrcamento`	adaptadores de entrada	Recebem a requisição, tratam a verificação prévia de CORS, extraem identificador e corpo, acionam o caso de uso e devolvem a resposta
Casos de uso `create`, `list`, `update`, `delete`	application	Aplicam o domínio quando necessário e delegam a persistência ao repositório recebido por injeção
`domain/orcamento.js`	domain	Valida o formato do orçamento, descarta a chave técnica do corpo, gera o código de negócio e preenche situação e datas
`mongoOrcamentoRepository`	infrastructure	Traduz as operações do repositório em comandos do driver MongoDB e resolve o filtro por chave técnica ou de negócio
`shared/infra/mongodb/client.js`	infraestrutura compartilhada	Resolve a configuração de conexão, cria o cliente na primeira invocação e o reaproveita nas seguintes
`orcamento.presenter.js`	adaptador de saída	Serializa o identificador técnico e converte erro de domínio em código HTTP
`shared/http/response.js`	adaptador de saída	Monta corpo JSON, cabeçalhos de CORS e identificação da versão da função

Operação	Método e rota	Entrada	Respostas
Consultar	GET `/api/GetOrcamentos`	—	200 com a lista ordenada por atualização; 500 em falha de infraestrutura
Criar	POST `/api/CreateOrcamento`	Orçamento em JSON	201 com o orçamento gravado; 400 em payload inválido; 500 em falha
Atualizar	PUT `/api/UpdateOrcamento`	Identificador na consulta ou no corpo, mais o orçamento	200 com o orçamento atualizado; 400 sem identificador; 404 quando não há correspondência; 500 em falha
Remover	DELETE `/api/DeleteOrcamento`	Identificador na consulta	200 com o identificador removido; 400 sem identificador; 404 quando não há correspondência; 500 em falha

5.4 Nível 3 de decomposição — código (C4 nível 4)

O nível 4 do C4 Model é apresentado com três notações UML complementares: um diagrama de componentes com interfaces fornecidas e requeridas, que mostra como as duas pontas do sistema se encaixam; e diagramas de classes do modelo de domínio, da fatia do frontend e da API.

5.4.1 Diagrama de componentes UML

O diagrama destaca as interfaces que atravessam as fronteiras: a porta de casos de uso consumida pela interface gráfica, a porta de gateway implementada pelo adaptador HTTP, a fronteira HTTP entre os contêineres e a porta de repositório implementada sobre o driver do banco.

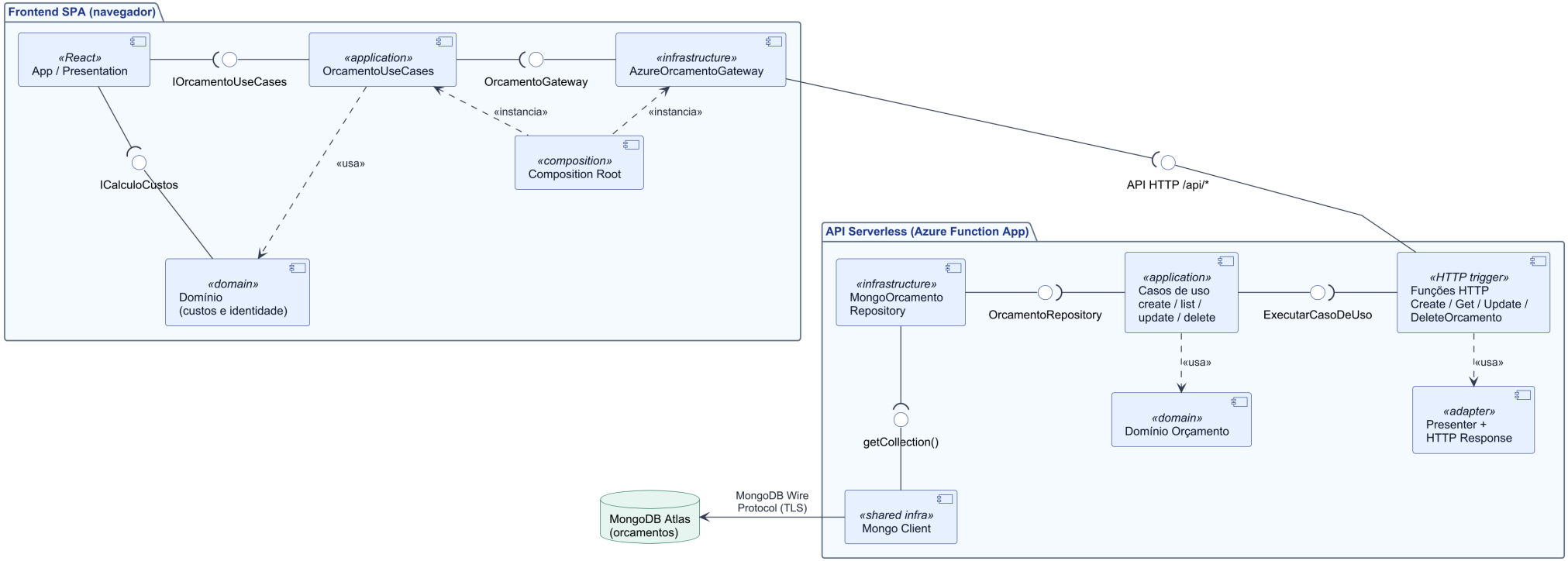


Figura 8 — C4 Model — Nível 4: diagrama de componentes UML com interfaces fornecidas e requeridas

5.4.2 Diagrama de classes UML — modelo de domínio

O modelo de domínio é compartilhado conceitualmente pelas duas pontas: o orçamento é o agregado, a operação é a entidade que compõe a folha de processo e o desenho técnico é um objeto de valor. O resultado do cálculo é um objeto de valor derivado, nunca persistido.

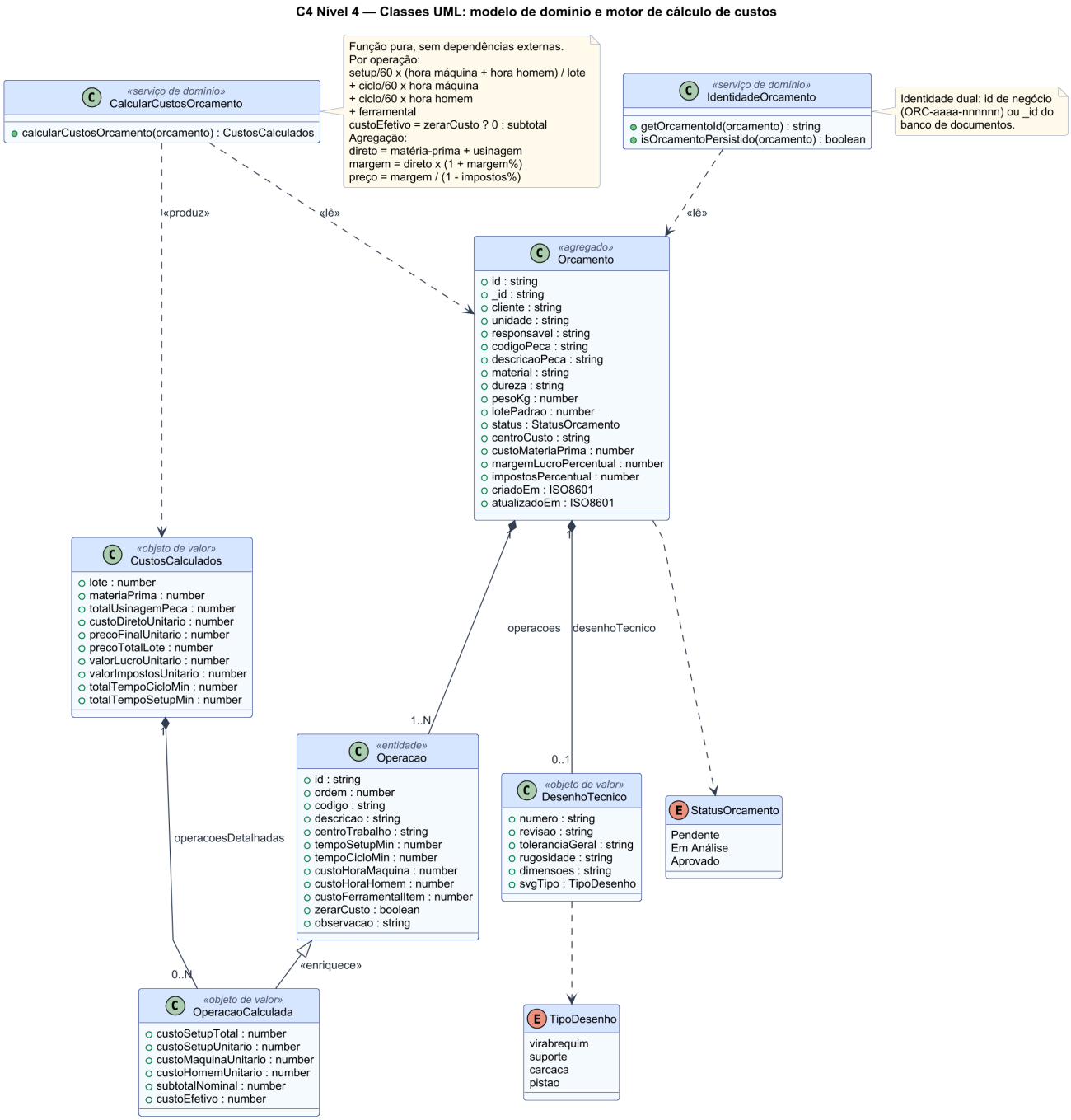


Figura 9 — C4 Model — Nível 4: classes do modelo de domínio e do motor de cálculo de custos

5.4.3 Diagrama de classes UML — fatia do frontend

No frontend, a porta `OrcamentoGateway` separa os casos de uso do mundo externo. O adaptador `AzureOrcamentoGateway` a implementa e é o único módulo que conhece URLs, chave de função e variáveis de ambiente. O composition root instancia os dois e publica a configuração para a apresentação, que por isso não precisa importar infraestrutura.

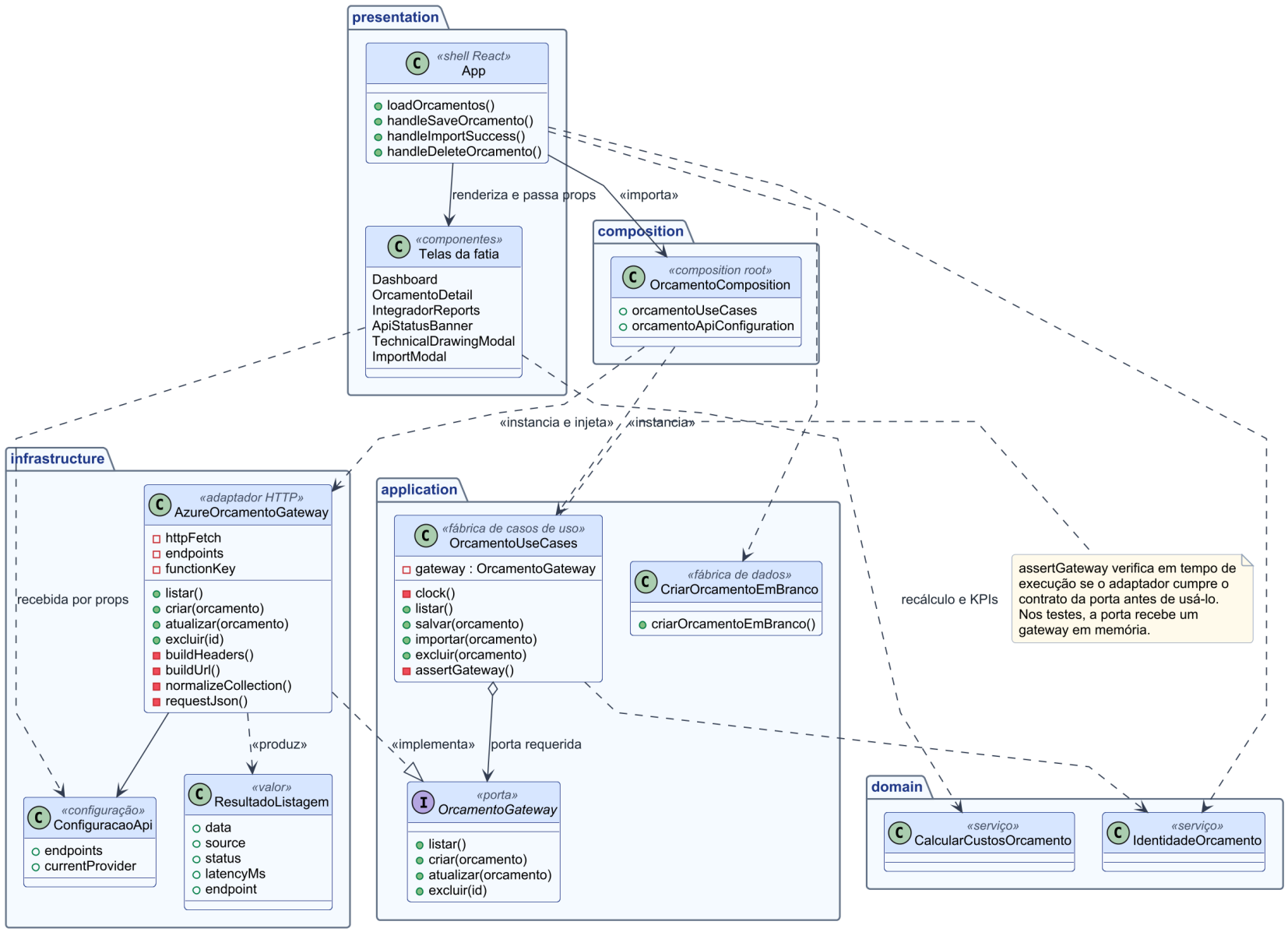


Figura 10 — C4 Model — Nível 4: classes de casos de uso, porta e adaptadores do frontend

5.4.4 Diagrama de classes UML — API serverless

Na API, cada arquivo de função atua como composition root da sua fatia: injeta o repositório concreto no caso de uso uma única vez, no carregamento do módulo. Os casos de uso recebem a porta `Orcamento-Repository` e, quando a operação exige regra, chamam o domínio. O apresentador e o módulo de resposta HTTP ficam na borda de saída.

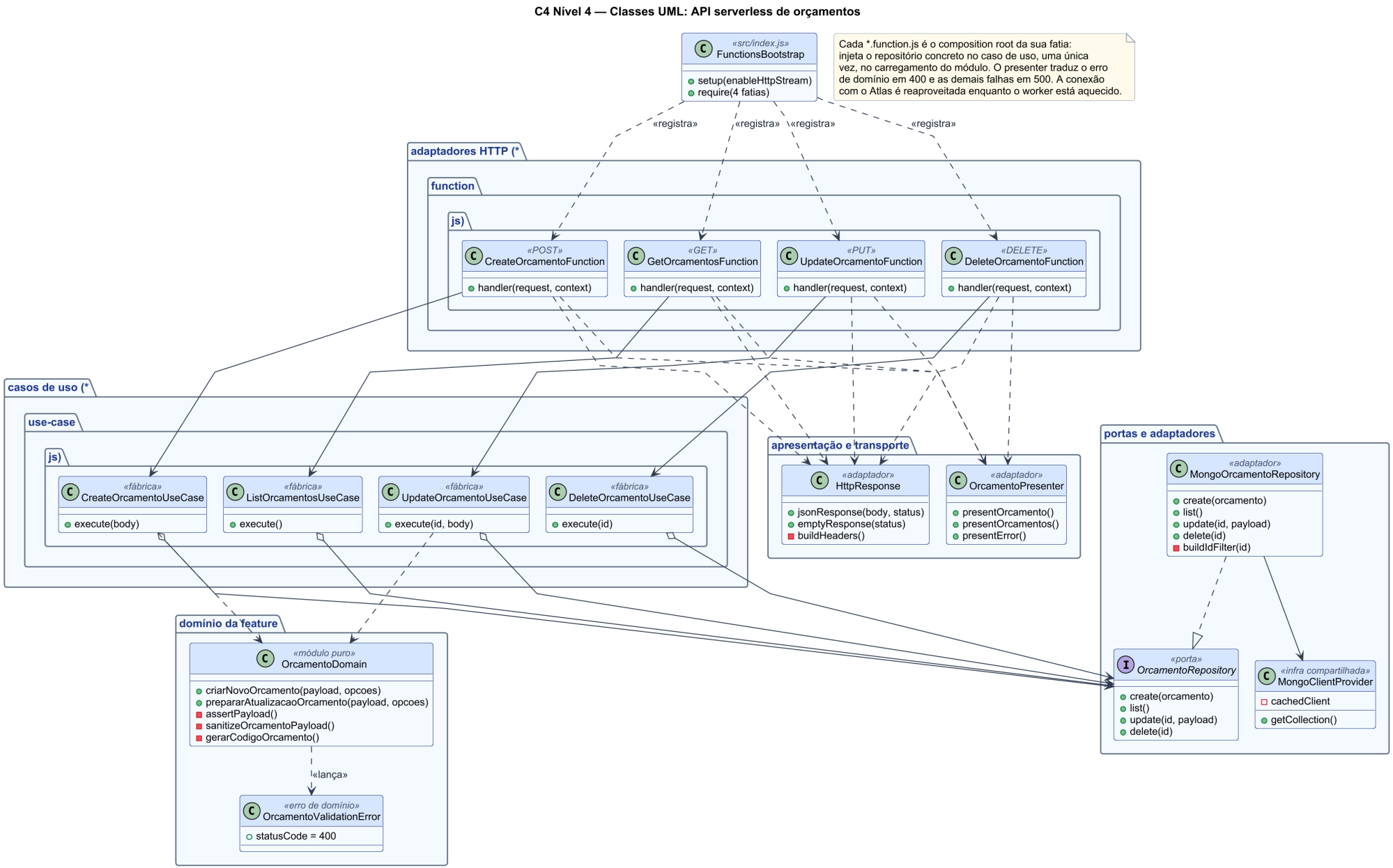


Figura 11 — C4 Model — Nível 4: classes da API serverless de orçamentos

6. Visão de Runtime (Runtime View)

Os quatro cenários a seguir foram escolhidos porque exercitam todas as fronteiras da arquitetura: a leitura ponta a ponta, o recálculo puramente local que caracteriza o produto, a gravação com validação no servidor e a geração de saídas para fora do sistema.

6.1 Consulta de orçamentos e indicadores

Ao abrir a aplicação, o shell aciona o caso de uso de listagem, que atravessa a porta até o adaptador HTTP. A função de consulta delega ao caso de uso, que usa o repositório para ler a coleção ordenada pela última atualização. O adaptador devolve ao shell um envelope com os dados e a latência medida, que alimenta o banner de situação. O dashboard calcula os custos de cada item para compor os indicadores.

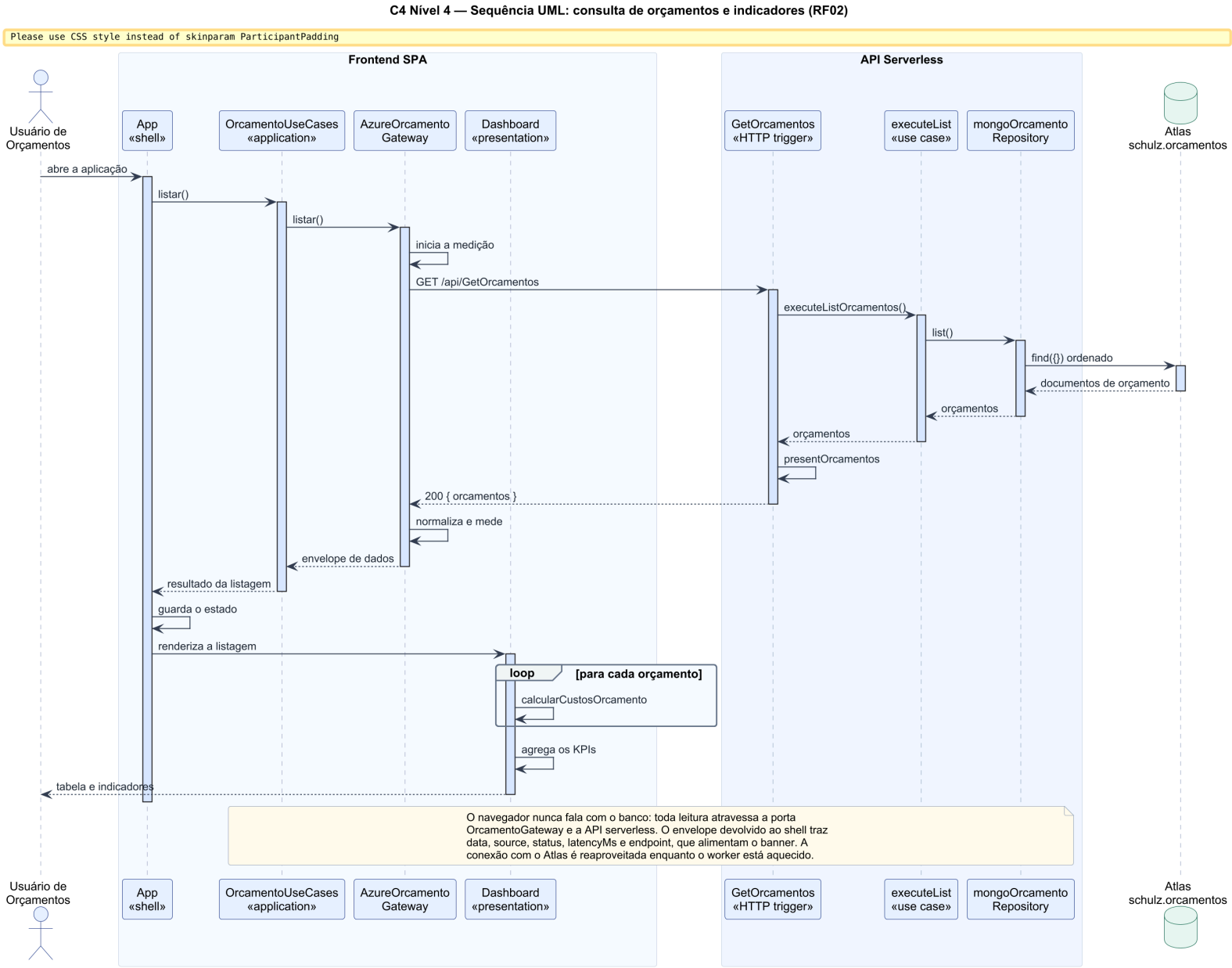


Figura 12 — Sequência UML: consulta de orçamentos e composição dos indicadores (RF02)

6.2 Flag Zerar Custo e recálculo instantâneo

Este é o cenário que justifica a escolha de manter o motor de cálculo no cliente. Ao acionar a flag em uma operação, o formulário recalcula o orçamento inteiro chamando a função de domínio: o custo efetivo da operação zerada passa a ser nulo, mas o subtotal nominal e os tempos de setup e de ciclo permanecem visíveis para o planejamento. Nenhuma chamada de rede acontece.

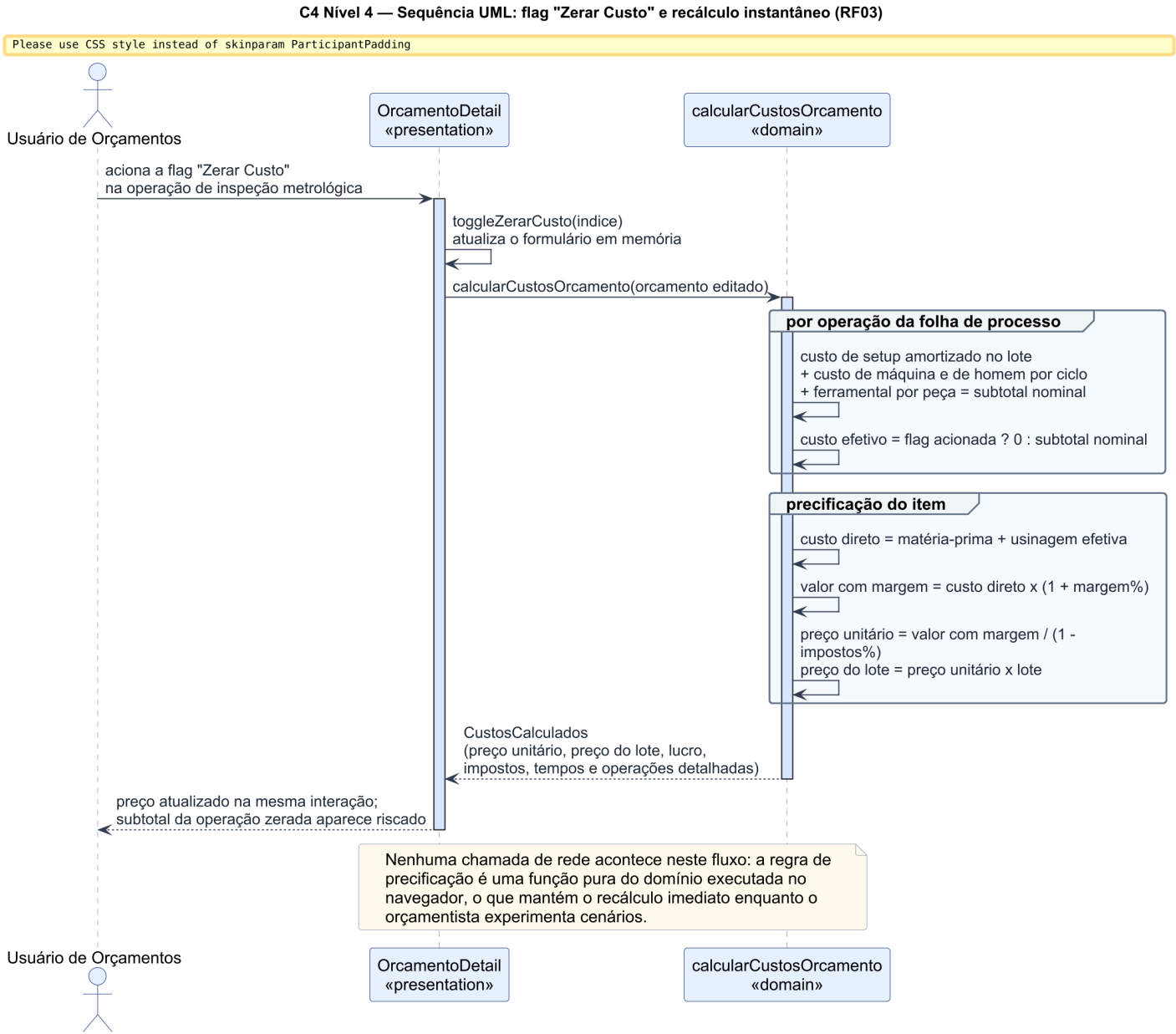


Figura 13 — Sequência UML: flag Zerar Custo e recálculo imediato do preço (RF03)

6.3 Gravação do orçamento recalculado

Ao salvar, o caso de uso decide entre criar e atualizar conforme a identidade do orçamento e registra a data de atualização. No servidor, o identificador é resolvido na consulta ou no corpo, o domínio valida o formato e descarta a chave técnica, e o repositório grava devolvendo o documento já atualizado. Depois da gravação, o shell recarrega a listagem: o banco permanece a única fonte da verdade.

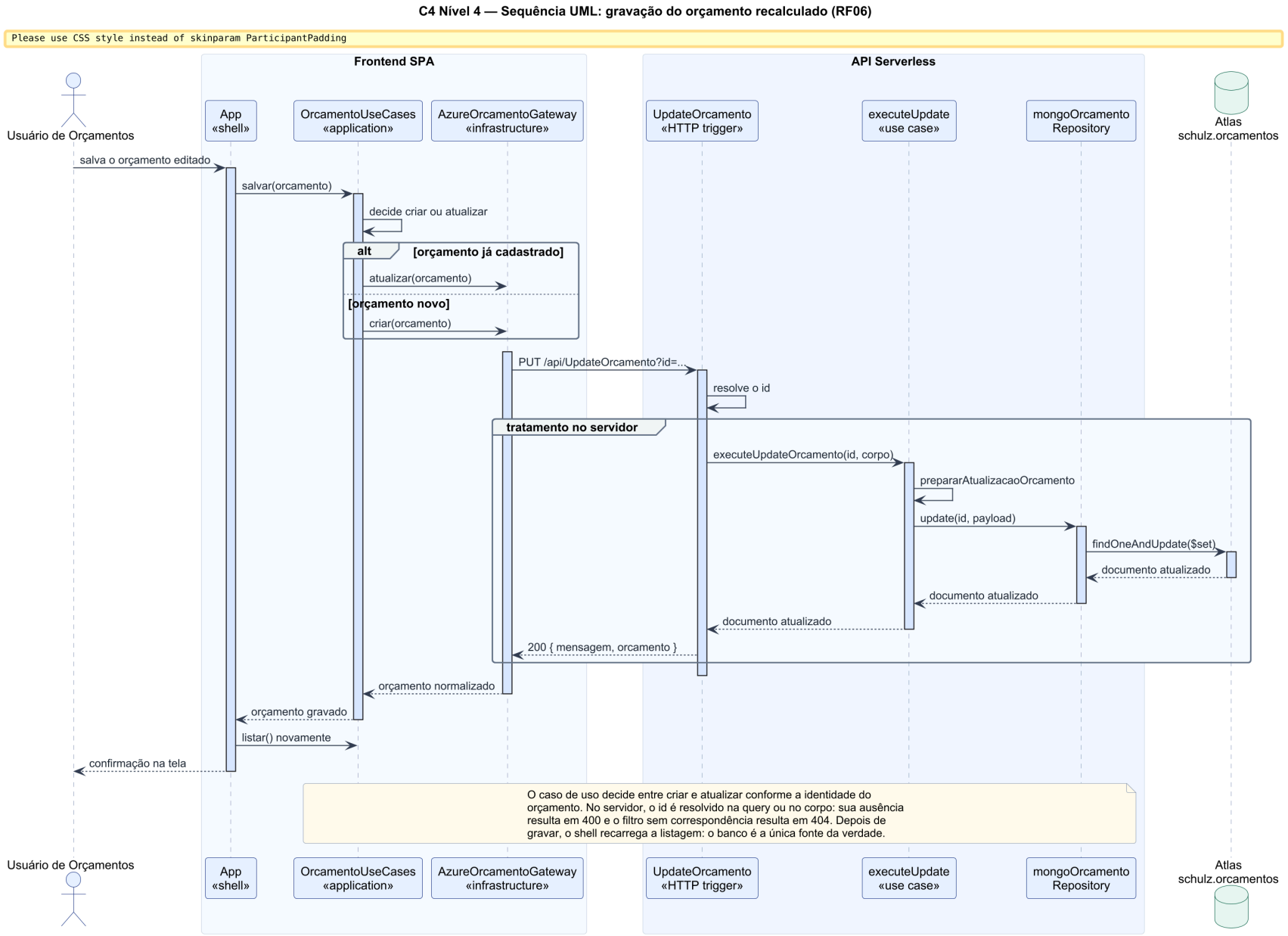


Figura 14 — Sequência UML: gravação do orçamento com validação de domínio (RF06)

6.4 Proposta comercial e exportação estruturada

A proposta é uma projeção derivada: é recalculada a partir do agregado no momento da exibição e não é persistida. A emissão usa o serviço de impressão do sistema operacional, com a página formatada em A4. A exportação monta o payload estruturado em memória e o entrega como arquivo, para consumo pelos sistemas corporativos do cliente.

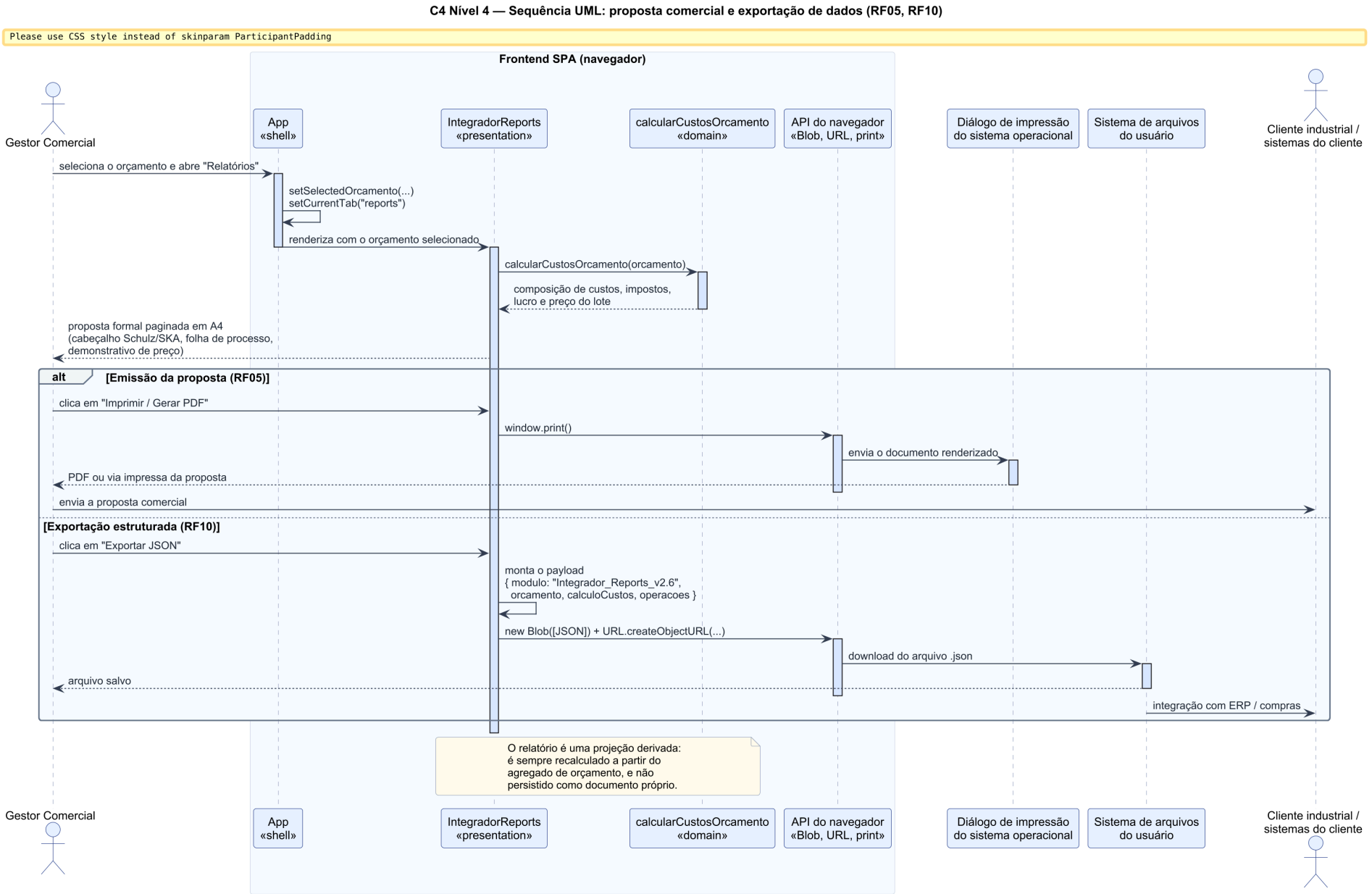


Figura 15 — Sequência UML: emissão da proposta comercial e exportação em JSON (RF05, RF10)

7. Visão de Implantação (Deployment View)

A solução ocupa quatro ambientes de execução independentes: a estação do desenvolvedor, a infraestrutura do GitHub (esteira e hospedagem estática), o Function App na Azure e o cluster gerenciado do MongoDB Atlas. Essa separação é o que permite publicar frontend e API em ritmos diferentes.

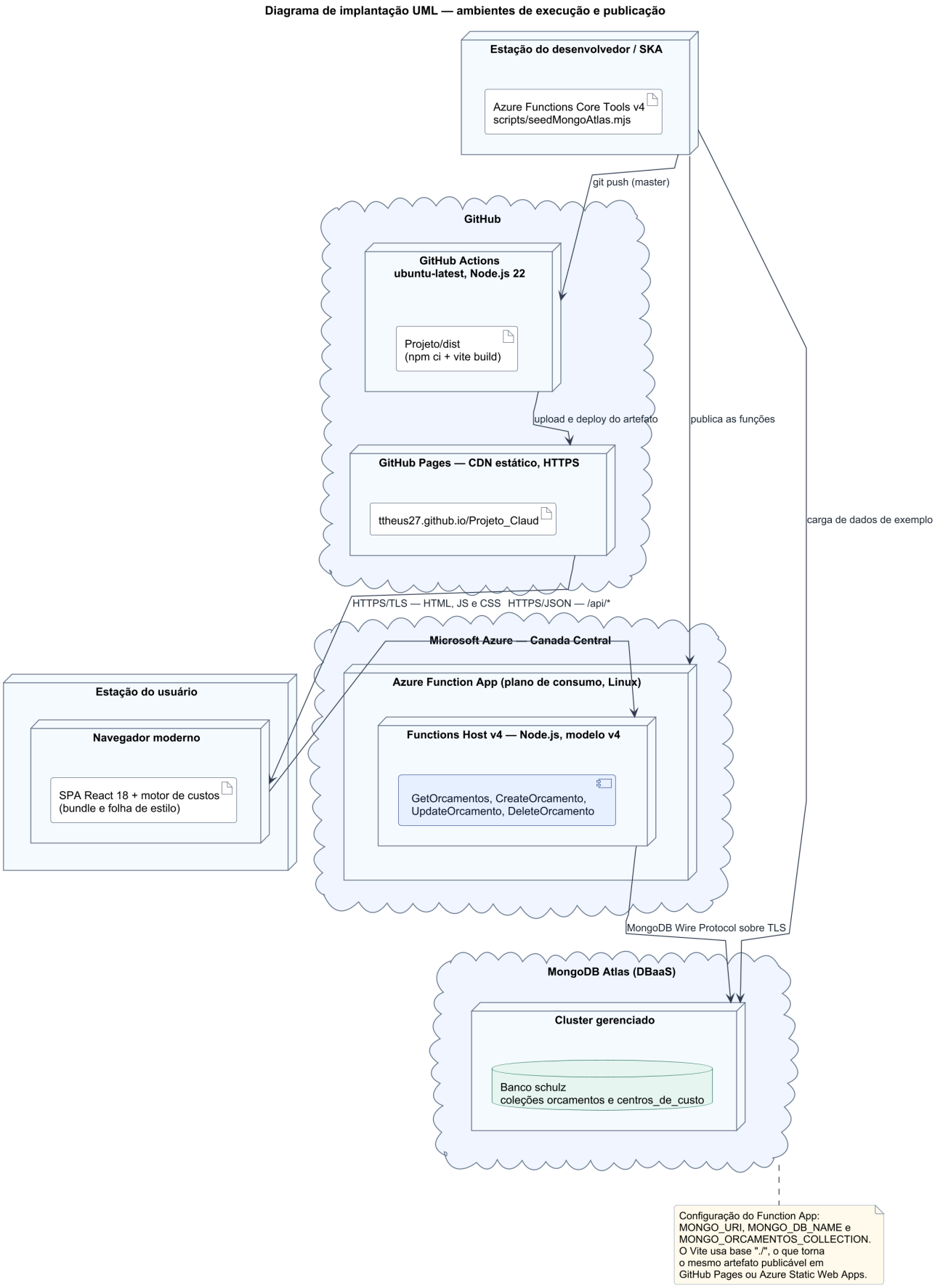


Figura 16 — Diagrama de implantação UML: nós de execução, artefatos e canais

Nó de execução	Artefato implantado	Observações de operação
Navegador do usuário	HTML, bundle JavaScript com as regras de domínio e folha de estilo compilada	Sem instalação: a atualização chega na próxima carga da página
GitHub Actions (ubuntu-latest, Node.js 22)	Resultado do build do frontend	Disparo por envio para a ramificação principal ou execução manual; cache de dependências pelo arquivo de bloqueio
GitHub Pages	Site estático publicado	Entrega por CDN com HTTPS; caminhos relativos permitem trocar de provedor sem reescrever o código
Azure Function App (plano de consumo, Linux)	Quatro funções HTTP no Functions Host v4	Escala por invocação; configuração em variáveis de ambiente do próprio recurso; telemetria de invocações e erros
MongoDB Atlas	Banco `schulz` com as coleções `orcamentos` e `centros_de_custo`	Cluster gerenciado com backup do provedor; acesso restrito por credencial e lista de origens permitidas
Estação do administrador	Ferramentas oficiais da Azure e script de carga	Publicação da API e recarga de dados de exemplo quando necessário

7.1 Configuração por ambiente

Parâmetro	Onde é definido	Para que serve
`MONGO_URI`	Configuração do Function App e arquivo local não versionado	String de conexão do cluster; sua ausência interrompe a invocação com erro de configuração
`MONGO_DB_NAME`	Configuração do Function App	Nome do banco; padrão `schulz`
`MONGO_ORCAMENTOS_COLLECTION`	Configuração do Function App	Nome da coleção de orçamentos; padrão `orcamentos`
`VITE_AZURE_FUNCTIONS_BASE_URL`	Arquivo de ambiente do frontend, em tempo de build	Aponta a SPA para a API local ou para outro ambiente
`VITE_AZURE_GET|CREATE|UPDATE|DELETE_ORCAMENTO_URL`	Arquivo de ambiente do frontend	Permite sobrescrever endpoint por endpoint, útil em testes de integração
`VITE_AZURE_FUNCTION_KEY`	Arquivo de ambiente do frontend	Quando presente, acompanha as requisições no cabeçalho de chave de função

7.2 Procedimento de publicação

1. O desenvolvedor envia a alteração para a ramificação principal do repositório.
2. A esteira do GitHub Actions instala as dependências do frontend, gera o build e publica o artefato no GitHub Pages.
3. O administrador publica a API com as ferramentas oficiais da Azure e confere as variáveis de ambiente do Function App.
4. Quando o ambiente é novo, o script de carga cria as coleções, cria os índices únicos e grava os dados de exemplo de forma idempotente.

5. A verificação final é feita pela própria aplicação: o banner de situação mostra o endpoint em uso e a latência da consulta.

8. Conceitos Transversais (Crosscutting Concepts)

Os conceitos desta seção valem para todo o sistema, independentemente do bloco de construção: o modelo de dados, a regra de precificação, o contrato HTTP, a configuração, a segurança, a observabilidade e a estratégia de testes.

8.1 Modelo de domínio e de dados

O orçamento é tratado como **agregado**: a peça, seus metadados comerciais, a folha de processo e o desenho técnico formam uma unidade de consistência e são lidos e gravados juntos. O diagrama de entidades e relacionamentos abaixo representa esse modelo no banco de documentos, incluindo o que é embutido, o que é coleção própria e o que é derivado em tempo de exibição.

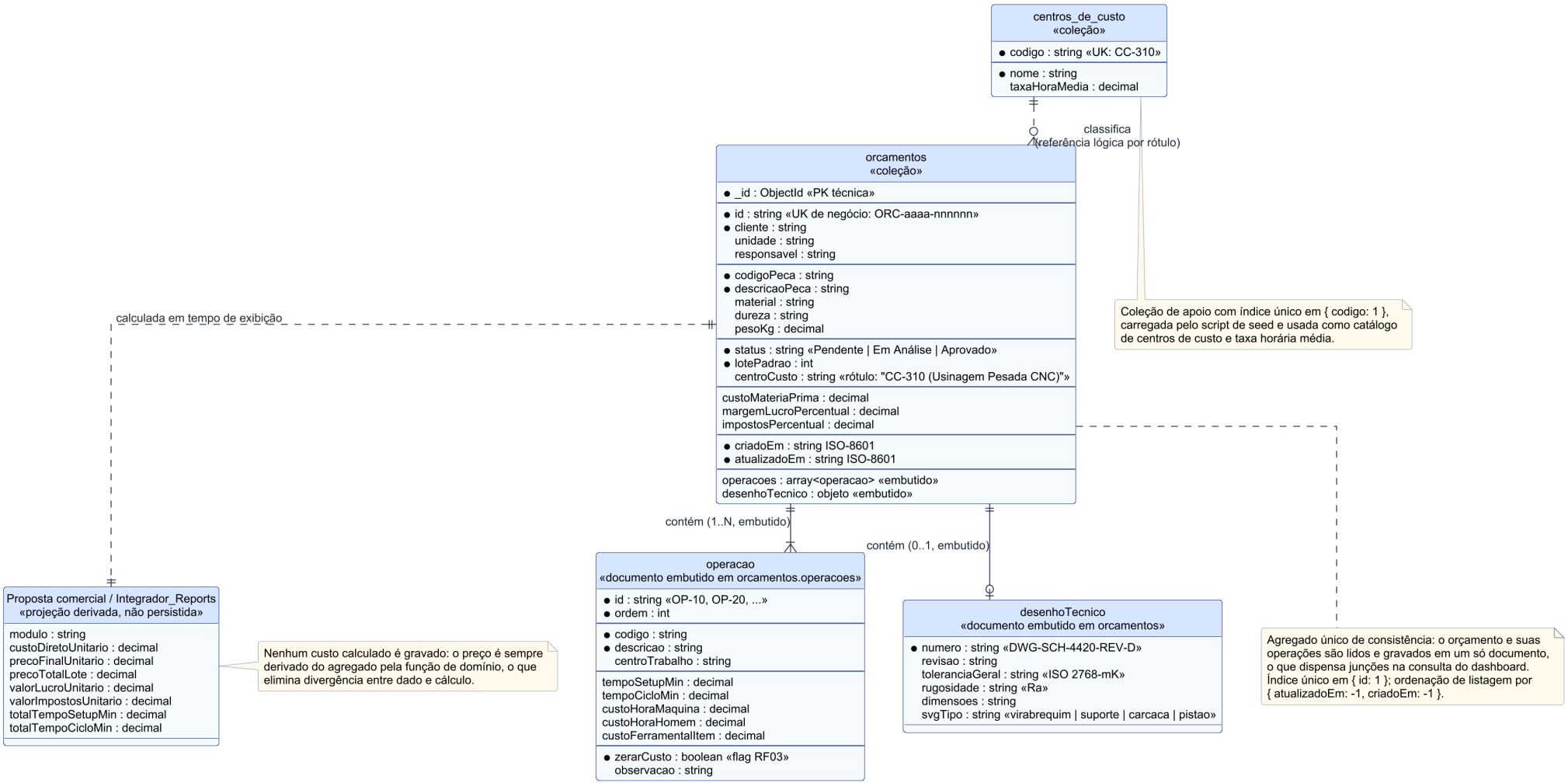


Figura 17 — Diagrama de Entidades e Relacionamentos do banco de documentos `schulz`

Estrutura	Tipo	Chaves e índices	Observação
`orcamentos`	Coleção	Chave técnica `_id`; índice único em `id` (código de negócio `ORC-aaaa-nnnnnn`)	Documento raiz do agregado; listagem ordenada por `atualizadoEm` e `criadoEm`
`operacoes`	Lista embutida em `orcamentos`	Identificador sequencial da operação (`OP-10`, `OP-20`, …) e campo de ordem	Cardinalidade 1..N; cada item guarda tempos, custos horários, ferramental e a flag de custo zerado
`desenhoTecnico`	Objeto embutido em `orcamentos`	Número e revisão do desenho	Cardinalidade 0..1; guarda tolerância geral, rugosidade, dimensões e o tipo de representação vetorial
`centros_de_custo`	Coleção	Índice único em `codigo` (`CC-310`, `CC-220`, …)	Catálogo de centros de custo e taxa horária média, carregado pelo script de carga
Composição de preço	Projeção derivada	—	Calculada pela função de domínio a cada exibição; não ocupa espaço no banco e não pode divergir do dado

> A relação entre orçamento e centro de custo é **lógica**: o orçamento guarda o rótulo do centro de custo, e o catálogo existe para alimentar o formulário e padronizar a taxa horária. Essa escolha mantém a leitura do dashboard em uma única consulta, sem junção.

8.2 Regra de precificação

A regra é implementada como função pura: recebe o orçamento e devolve a composição completa do preço, sem efeito colateral e sem dependência de infraestrutura. É executada a cada renderização do formulário, da proposta e do dashboard.

```
Por operação da folha de processo:
  custoSetupTotal      = (tempoSetupMin / 60) x (custoHoraMaquina + custoHoraHomem)
  custoSetupUnitario   = custoSetupTotal / lote            // setup amortizado no lote
  custoMaquinaUnitario = (tempoCicloMin / 60) x custoHoraMaquina
  custoHomemUnitario   = (tempoCicloMin / 60) x custoHoraHomem
  subtotalNominal      = custoSetupUnitario + custoMaquinaUnitario
                       + custoHomemUnitario + custoFerramentalItem
  custoEfetivo         = zerarCusto ? 0 : subtotalNominal  // flag RF03

Agregação e precificação do item:
  totalUsinagemPeca    = soma dos custos efetivos das operações
  custoDiretoUnitario  = custoMateriaPrima + totalUsinagemPeca
  valorComMargem       = custoDiretoUnitario x (1 + margemLucroPercentual / 100)
  precoFinalUnitario   = valorComMargem / (1 - impostosPercentual / 100)
  precoTotalLote       = precoFinalUnitario x lote
  valorLucroUnitario   = valorComMargem - custoDiretoUnitario
  valorImpostosUnitario= precoFinalUnitario - valorComMargem
```

- O **setup é amortizado no lote**: quanto maior o lote, menor o peso do tempo de preparação no preço unitário, o que reproduz a prática de orçamentação da planta.
- O **ferramental é custo por peça**, porque representa consumo de inserto e de rebolo por item produzido.

- A **flag Zerar Custo** anula apenas o custo efetivo da operação. O subtotal nominal continua disponível e é exibido riscado, e os tempos de setup e de ciclo seguem somando nos totais, pois a operação ocupa máquina mesmo quando não é cobrada.
- Os impostos entram por **dentro do preço**: o valor com margem é dividido pelo complemento da alíquota, de modo que o preço final absorva a carga tributária.
- Todos os campos numéricos são saneados na entrada da função, e o lote nulo é tratado como unitário para que o cálculo nunca divida por zero.

8.3 Contrato HTTP, erros e CORS

Todas as respostas da API têm corpo JSON e carregam a identificação da versão da função, o que facilita conferir qual build está atendendo em produção. A tradução de falhas é centralizada no apresentador: erro de validação de domínio responde 400 e qualquer outra falha responde 500, sempre com a mensagem no campo de erro.

Situação	Código	Corpo
Consulta bem-sucedida	200	`{ orcamentos: [...] }`
Criação bem-sucedida	201	`{ mensagem, orcamento }` com o documento gravado
Atualização bem-sucedida	200	`{ mensagem, orcamento }` com o documento já atualizado
Remoção bem-sucedida	200	`{ mensagem, id }`
Formato de orçamento inválido ou identificador ausente	400	`{ error: "<motivo>" }`
Orçamento inexistente na atualização ou na remoção	404	`{ error: "Orçamento não encontrado." }`
Falha de infraestrutura, inclusive configuração ausente	500	`{ error: "<motivo>" }` e registro na telemetria
Verificação prévia do navegador	200	Resposta vazia com os cabeçalhos de CORS

O frontend tolera variações do envelope de resposta: aceita a coleção como lista direta ou dentro de um campo de dados, e o mesmo vale para o documento individual. Essa normalização fica contida no adaptador, o que isola o restante da aplicação de mudanças no formato da API.

8.4 Configuração, segredos e ambientes

- A API lê toda a configuração de variáveis de ambiente; a ausência da string de conexão interrompe a invocação com mensagem explícita, em vez de falhar silenciosamente.
- O frontend resolve os endpoints em tempo de build, com valor padrão apontando para o ambiente publicado e possibilidade de sobrescrita por variável, inclusive endpoint por endpoint.
- Arquivos de ambiente e de configuração local ficam fora do versionamento; o repositório guarda apenas os modelos de exemplo.
- A chave de função, quando configurada, acompanha a requisição em cabeçalho próprio, sem aparecer na URL.

8.5 Segurança e transporte

- Todo o tráfego é cifrado: HTTPS entre navegador e hospedagem, HTTPS entre navegador e API, e TLS com Stable API v1 entre a API e o cluster do banco.
- O navegador não possui credencial de banco: a API é a única fronteira com acesso ao cluster, e o acesso é restrito por credencial e por lista de origens permitidas no provedor.
- O domínio da API descarta a chave técnica enviada no corpo, o que impede que uma requisição sobrescreva o identificador interno do documento.
- As respostas declaram explicitamente os métodos e cabeçalhos aceitos, e a verificação prévia do navegador é respondida pela própria função.
- O perfil de acesso por tipo de usuário (RF08) é posicionado na camada de aplicação, onde a autorização acompanha o caso de uso e permanece independente do mecanismo de autenticação adotado na borda.

8.6 Observabilidade

- Cada função registra o início da operação e o detalhe da falha, com os registros coletados pela telemetria do provedor e amostragem configurada para conter volume.
- A resposta carrega a identificação da versão da função, o que permite correlacionar comportamento observado com build publicado.
- O adaptador do frontend mede a latência de cada consulta e a expõe no banner de situação, dando ao usuário um sinal direto da saúde da integração.
- A marcação temporal de criação e atualização em cada documento permite reconstruir a ordem das alterações do orçamento.

8.7 Estratégia de testes

A suíte roda com Vitest e cobre três níveis: a regra de negócio, a orquestração dos casos de uso e a interface. Acima deles, os **testes unitários de arquitetura** verificam a própria estrutura do código: leem os módulos, extraem as importações e falham se uma camada passar a depender de quem não deveria.

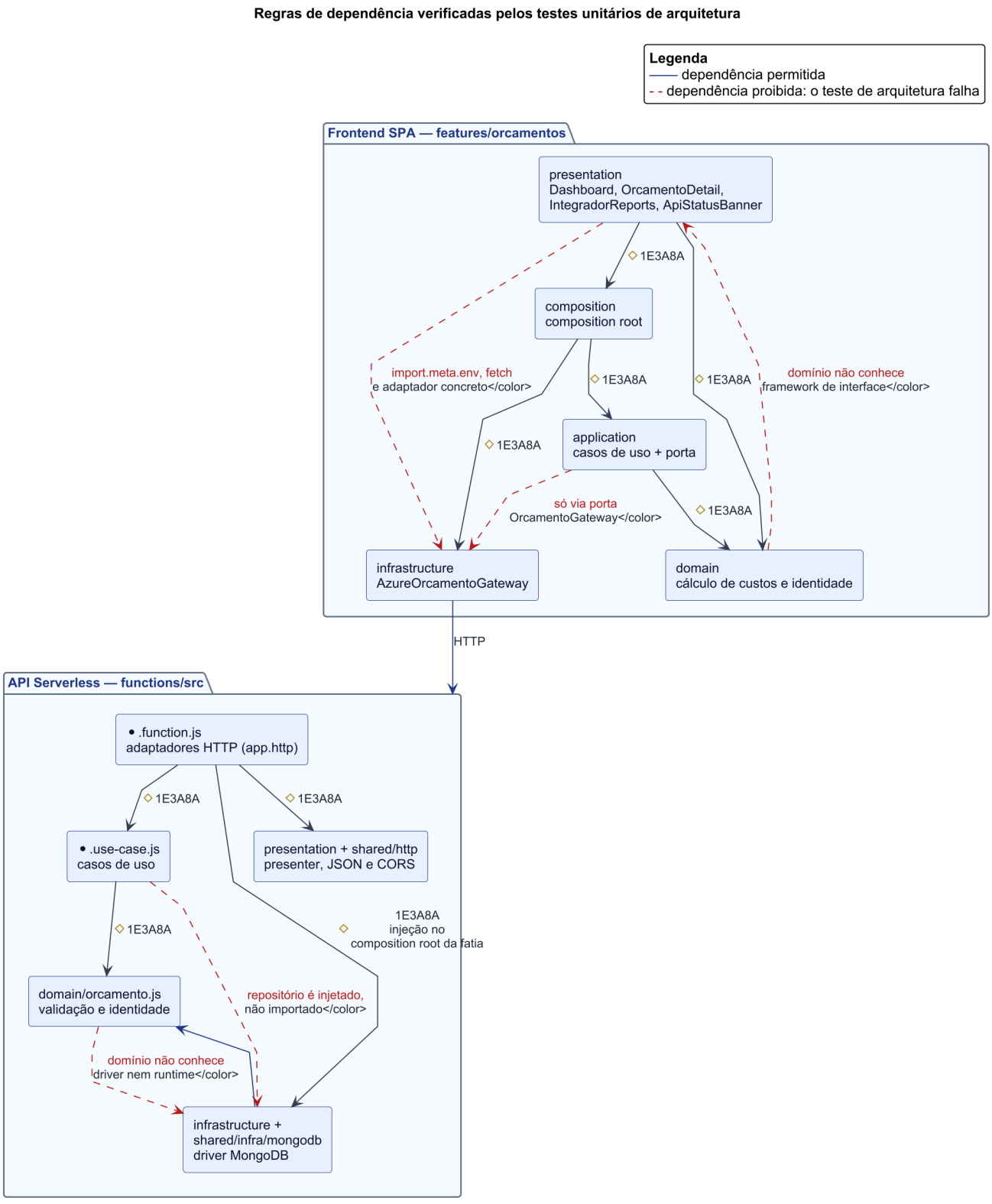


Figura 18 — Regras de dependência verificadas pelos testes unitários de arquitetura

Nível	O que é verificado	Dublês e ferramentas
Domínio	Composição do preço a partir de setup, ciclo, ferramental, margem e impostos; comportamento da flag de custo zerado, que preserva tempos e subtotal nominal	Nenhum: a função é pura
Aplicação	Criação de orçamento como pendente com datas, atualização preservando a data de criação e recusa de exclusão sem identificação	Gateway em memória e relógio controlado
Apresentação	Filtro do dashboard devolvendo somente o orçamento correspondente ao código pesquisado	Testing Library com ambiente de navegador simulado
Arquitetura (frontend)	Domínio sem dependência externa; aplicação dependente apenas do domínio; apresentação sem acesso direto à infraestrutura; setas apontando para dentro; configuração de ambiente e cliente HTTP restritos à infraestrutura; adaptador concreto instanciado só no composition root; nenhuma fatia importando outra	Leitura do código-fonte pelo próprio teste
Arquitetura (API)	Domínio sem conhecimento do runtime de funções e do driver; casos de uso recebendo o repositório por injeção; rotas declaradas apenas nos adaptadores de função; driver do banco restrito à infraestrutura; fatias independentes entre si	Leitura do código-fonte pelo próprio teste

> Os cenários são escritos em DADO/QUANDO/ENTÃO e em linguagem de domínio, para que descrevam comportamento observável em vez de detalhe de implementação. A relação completa dos cenários acompanha o repositório.

8.8 Experiência de uso e desempenho percebido

- A navegação acontece por estado no shell, sem recarregar a página e sem depender de rota no servidor, o que combina com a hospedagem estática.
- O recálculo local elimina a espera entre a alteração de um tempo e a atualização do preço, que é o gesto mais repetido pelo usuário.
- Depois de cada gravação a base é recarregada, de modo que a tela sempre reflete o que está persistido.
- A proposta é formatada para impressão em A4, e a exportação entrega o mesmo conteúdo em formato estruturado para integração.

9. Decisões de Arquitetura (Architecture Decisions)

As decisões estão registradas no formato de ADR (Architecture Decision Record). As quatro primeiras explicam a forma do sistema; as quatro seguintes registram a decisão de não adotar, neste escopo, os demais estilos arquiteturais estudados na disciplina; as duas últimas tratam de garantia estrutural e de publicação.

9.1 Decisões estruturantes

ADR-01 — API serverless em Azure Functions

Campo	Conteúdo
Situação	Aceita e implementada
Contexto	O uso do sistema é concentrado no horário comercial da equipe de orçamentos, com picos curtos durante a montagem de propostas. Não há equipe de infraestrutura dedicada e o orçamento operacional é restrito às camadas gratuitas da plataforma.
Decisão	Expor a camada de dados como quatro funções HTTP em Azure Functions v4, uma por operação de orçamento, em plano de consumo, com a conexão de banco reaproveitada entre invocações do mesmo worker.
Consequências	Escala automática por invocação e ausência de servidor para administrar. Em contrapartida, a primeira invocação após um período de inatividade carrega o custo de partida a frio, e o estado precisa viver inteiramente no banco, o que a arquitetura já pressupõe.
Alternativas consideradas	Servidor Node.js contínuo em máquina virtual ou contêiner, descartado pelo custo fixo e pelo esforço de operação; acesso direto do navegador ao banco, descartado por expor credencial no cliente.

ADR-02 — Clean Architecture com Ports & Adapters nas duas pontas

Campo	Conteúdo
Situação	Aceita e implementada
Contexto	A regra de precificação é o ativo mais sensível do produto: um erro nela se transforma em prejuízo comercial. Ela precisa ser testável de forma rápida e determinística, e precisa sobreviver a trocas de framework de interface e de provedor de nuvem.
Decisão	Organizar frontend e API em camadas com dependências apontando para dentro: domínio puro no centro, casos de uso orquestrando portas, adaptadores na borda e instanciação concentrada em um composition root. No frontend a porta é o gateway de orçamentos; na API é o repositório de orçamentos.
Consequências	O domínio é testado sem navegador, rede ou banco; os casos de uso aceitam dublês; trocar o adaptador HTTP ou o banco não altera a regra. O custo é a indireção adicional e a necessidade de disciplina nas importações, endereçada pela ADR-09.
Alternativas consideradas	Chamar a API diretamente dos componentes de interface, descartado por espalhar conhecimento de transporte pela tela; concentrar regra de negócio no backend, descartado porque inviabilizaria o recálculo imediato exigido pela meta de reatividade.

ADR-03 — Organização do código em fatias verticais (Vertical Slice)

Campo	Conteúdo
Situação	Aceita e implementada
Contexto	O sistema tem um domínio central muito denso (orçamento e folha de processo) e previsão de crescimento por funcionalidades distintas, como relatórios e importação. Uma estrutura organizada por tipo de arquivo obrigaria a percorrer a árvore inteira para cada mudança.
Decisão	Agrupar o código por funcionalidade: no frontend, a fatia `features/orcamentos` contém suas próprias camadas; na API, cada operação é uma fatia (`create` , `list` , `update` , `delete`) com adaptador e caso de uso próprios, compartilhando o núcleo de domínio, repositório e apresentação da feature.
Consequências	Mudança de funcionalidade fica contida em uma pasta, e a leitura do código acompanha o raciocínio do negócio. Em troca, exige vigilância contra dependência cruzada entre fatias, verificada por teste automatizado.
Alternativas consideradas	Camadas horizontais globais (`services` , `components` , `models`), descartadas pela dispersão; um único módulo monolítico por tela, descartado por misturar regra de negócio com apresentação.

ADR-04 — Banco de documentos com o orçamento como agregado único

Campo	Conteúdo
Situação	Aceita e implementada
Contexto	A tela de consulta precisa de orçamento, folha de processo e desenho técnico ao mesmo tempo, e o número de operações por peça é da ordem de dezenas. O cálculo de preço é sempre derivado do conjunto completo.
Decisão	Persistir o orçamento no MongoDB Atlas como documento único, com as operações e o desenho técnico embutidos, índice único no código de negócio e catálogo de centros de custo em coleção separada. Nenhum valor calculado é gravado.
Consequências	Leitura e gravação do agregado em uma única operação, sem junção, e consistência garantida dentro do documento. Em contrapartida, o centro de custo é referência lógica por rótulo, e relatórios analíticos entre orçamentos exigiriam agregação explícita.
Alternativas consideradas	Modelo relacional normalizado com tabelas de peça, operação e centro de custo, descartado por exigir junções para montar cada tela e por não haver demanda transacional entre agregados; gravar o preço calculado, descartado pelo risco de divergência entre dado e regra.

9.2 Estilos arquiteturais considerados e não adotados

Os quatro estilos a seguir foram estudados na disciplina e analisados para este sistema. Em cada caso a decisão foi **não adotar o estilo neste escopo**, e a ADR registra o critério objetivo que sustenta a escolha, as consequências assumidas e o gatilho que justificaria revisá-la. Decidir conscientemente contra um estilo é uma decisão de arquitetura como qualquer outra: evita complexidade sem contrapartida e deixa explícito o limite de validade do desenho atual.

ADR-05 — Microfrontend

Campo	Conteúdo
Situação	Aceita — estilo deliberadamente não adotado neste escopo

Campo	Conteúdo
Contexto	O estilo de microfrontend resolve o problema de múltiplas equipes publicando partes independentes de uma mesma interface. Aqui existe uma equipe e um domínio central, e o produto é publicado como artefato estático único.
Decisão	Manter um artefato de publicação único e tratar a **fatia vertical como unidade de extração**: a fatia de orçamentos já concentra suas telas, seus casos de uso, seu domínio e seu adaptador, e se comunica com o shell apenas por propriedades e callbacks.
Consequências	A publicação permanece simples e o tempo de carga continua baixo, sem orquestrador de módulos remotos. Em contrapartida, todo o frontend compartilha um mesmo ciclo de publicação. A fronteira que um microfrontend exigiria já está materializada no código: a fatia é autocontida e se comunica com o shell apenas por propriedades e callbacks, o que mantém o custo de uma eventual separação restrito à camada de publicação.
Alternativas consideradas	Federação de módulos com publicação independente por fatia, que traria ganho apenas com mais de uma equipe; iframes compostos no shell, descartados pela degradação de experiência e pelo custo de comunicação entre contextos.

ADR-06 — Backend for Frontend (BFF), microsserviços e banco por serviço

Campo	Conteúdo
Situação	Aceita — estilos deliberadamente não adotados neste escopo
Contexto	O conjunto **Backend for Frontend (BFF)**, microsserviços e banco por serviço endereça autonomia de equipes, ciclos de publicação independentes e isolamento de falhas em domínios amplos. O escopo atual é um domínio coeso, com um único consumidor (a SPA) e um único agregado transacional.
Decisão	Manter um serviço coeso de orçamentação, com a **função por operação como granularidade de implantação** e o agregado de orçamento como fronteira de dados. A SPA consome diretamente esse contrato, sem camada intermediária de composição.
Consequências	Evita-se a complexidade distribuída (consistência eventual, rastreamento entre serviços, duplicação de modelo) sem perder o benefício de publicar e escalar cada operação separadamente, já que a função por operação mantém a granularidade fina de implantação. Em contrapartida, as operações compartilham um mesmo banco e um mesmo ciclo de vida de serviço. As fatias de função delimitam a linha de corte caso o domínio venha a se dividir em serviços próprios.
Alternativas consideradas	Quebrar imediatamente em serviços de orçamento, custos e relatórios com bancos próprios, descartado por multiplicar superfície operacional sem necessidade de domínio; introduzir um BFF agora, descartado por não haver múltiplos consumidores nem agregação de fontes a fazer.

ADR-07 — API Gateway

Campo	Conteúdo
Situação	Aceita — estilo deliberadamente não adotado neste escopo
Contexto	Um gateway dedicado centraliza autenticação, limitação de uso, versionamento de contrato, políticas de CORS e observabilidade de borda. No escopo atual existe um único Function App, que já concentra roteamento sob um mesmo prefixo e responde às políticas de CORS.
Decisão	Expor o Function App diretamente, adotando o prefixo `/api` como **contrato de borda estável** e a chave de função como mecanismo de credencial suportado pelo cliente. O roteamento e as políticas de CORS ficam sob responsabilidade do próprio Function App.

Campo	Conteúdo
Consequências	Menos um componente para configurar, custear e monitorar, com o contrato de borda estável desde a primeira publicação. Em compensação, as políticas transversais residem na própria aplicação. Como o cliente depende apenas do prefixo, e não do endereço físico do serviço, um gateway pode ser posicionado à frente da API sem alterar o frontend nem romper o contrato publicado.
Alternativas consideradas	Azure API Management desde o início, descartado pelo custo e pela ausência, hoje, de políticas que justifiquem a camada; gateway próprio implementado como função de borda, descartado por reimplementar o que a plataforma oferece pronta.

ADR-08 — Arquitetura orientada a eventos (EDA)

Campo	Conteúdo
Situação	Aceita — estilo deliberadamente não adotado no fluxo principal
Contexto	A arquitetura orientada a eventos favorece desacoplamento temporal, reação assíncrona e múltiplos consumidores do mesmo fato de negócio. O fluxo central do Integrador, porém, é interativo e sincrônico: o usuário altera a folha de processo e precisa ver o preço e a confirmação de gravação imediatamente.
Decisão	Conduzir o fluxo principal em **requisição e resposta**, mantendo a consistência imediata exigida pela tela. O estilo de eventos fica restrito, por decisão, aos fatos de negócio que não pertencem à interação sincrônica do usuário — criação, recálculo e aprovação de orçamento — que são tratados como registros explícitos nos casos de uso.
Consequências	O caminho crítico fica simples de depurar e previsível para o usuário, sem consistência eventual em tela nem necessidade de barramento, esquema de mensagens ou política de reprocessamento. Em contrapartida, não há desacoplamento temporal entre a ação do usuário e a persistência. Como os casos de uso já nomeiam esses fatos de negócio de forma explícita, a publicação de eventos se resolveria na borda, sem tocar a regra de precificação.
Alternativas consideradas	Publicar comandos de orçamento em barramento e processá-los de forma assíncrona, descartado porque a tela depende da resposta para seguir; adotar separação entre leitura e escrita com projeções, descartado por acrescentar sincronização sem demanda analítica atual.

9.3 Garantia estrutural e publicação

ADR-09 — Testes unitários de arquitetura como guarda das fronteiras

Campo	Conteúdo
Situação	Aceita e implementada
Contexto	Decisões de camada e de fatia se perdem com o tempo quando dependem apenas de revisão manual. Em JavaScript não há compilador para barrar uma importação indevida, e o projeto é mantido por quatro pessoas em paralelo.
Decisão	Escrever testes que leem o código-fonte, extraem as importações e verificam as regras de dependência do frontend e da API. São doze cenários, executados junto com a suíte de negócio.
Consequências	A violação de fronteira falha na execução dos testes, e não na revisão. A documentação desta seção e as regras em código permanecem sincronizadas, pois ambas descrevem o mesmo conjunto de restrições. O custo é manter os testes atualizados quando uma fronteira mudar de propósito, o que é justamente o momento de decisão consciente.
Alternativas consideradas	Verificação por ferramenta externa de grafo de dependências, descartada por acrescentar dependência de build para o mesmo resultado; apenas convenção documentada, descartada por não resistir à erosão.

ADR-10 — Frontend como artefato estático com caminhos relativos

Campo	Conteúdo
Situação	Aceita e implementada
Contexto	A entrega acadêmica usa GitHub Pages, que publica em subcaminho e não oferece rota de fallback para aplicações de página única. A solução, porém, precisa caber também em Azure Static Web Apps sem alteração de código.
Decisão	Gerar o build com caminhos relativos, navegar por estado no shell em vez de rotas de URL e resolver os endpoints da API por variável de ambiente em tempo de build.
Consequências	O mesmo artefato funciona em qualquer provedor de hospedagem estática e a publicação é feita pela esteira automatizada. Em contrapartida, não há endereço direto por tela, o que é aceitável para um sistema de uso interno orientado a tarefa.
Alternativas consideradas	Roteador com histórico do navegador, descartado por exigir configuração de fallback indisponível na hospedagem adotada; roteamento por fragmento de URL, descartado por poluir o endereço sem benefício para o fluxo atual.

10. Requisitos de Qualidade (Quality Requirements)

10.1 Árvore de qualidade

A árvore relaciona os atributos priorizados em 1.2 com os refinamentos que a arquitetura precisa sustentar. A coluna de prioridade indica a importância para o negócio e a de sensibilidade indica o quanto a decisão arquitetural afeta o atributo.

Atributo (ISO/IEC 25010)	Refinamento	Prior.	Sensib.
Eficiência de desempenho	Recálculo completo do preço durante a edição da folha de processo	Alta	Alta
Eficiência de desempenho	Tempo de resposta da consulta de orçamentos, incluindo partida a frio da função	Média	Alta
Manutenibilidade	Regra de negócio testável de forma isolada e determinística	Alta	Alta
Manutenibilidade	Mudança de funcionalidade contida em uma fatia vertical	Alta	Alta
Manutenibilidade	Fronteiras entre camadas verificadas automaticamente	Média	Alta
Confiabilidade	Falha de integração comunicada ao usuário sem perder o trabalho em tela	Média	Média
Adequação funcional	Preço reproduzível: mesma entrada sempre produz a mesma composição de custos	Alta	Média
Portabilidade	Publicação do frontend em mais de um provedor sem alteração de código	Média	Alta
Segurança	Credencial de banco restrita ao lado servidor	Alta	Alta
Usabilidade	Proposta comercial pronta para envio, sem edição manual posterior	Média	Baixa

10.2 Cenários de qualidade

Cada cenário descreve estímulo, resposta esperada e medida de aceitação, no formato usado em avaliações de arquitetura.

Id	Cenário	Resposta esperada	Medida
CQ-01	O orçamentista aciona a flag Zerar Custo em uma operação de inspeção de um orçamento com dez operações	O preço unitário, o preço do lote, o lucro e os impostos são recalculados e exibidos, com os tempos preservados	Atualização percebida como imediata, sem chamada de rede no cenário
CQ-02	O usuário abre a aplicação após longo período de inatividade da função	A listagem é apresentada e o banner informa o endpoint e a latência medida	Latência exibida ao usuário; falha resulta em aviso explícito em vez de tela vazia
CQ-03	Um desenvolvedor altera a fórmula de amortização do setup	Os testes de domínio apontam a mudança de comportamento antes da publicação	Suíte completa executada em poucos segundos, sem navegador, rede ou banco
CQ-04	Um desenvolvedor importa o adaptador HTTP dentro de um componente de tela	A suíte falha indicando a fronteira violada	Pelo menos um dos doze cenários de arquitetura falha na execução

Id	Cenário	Resposta esperada	Medida
CQ-05	A API recebe um corpo de orçamento com o campo de operações em formato inválido	A requisição é rejeitada com código 400 e mensagem de domínio, sem gravar nada	Nenhuma alteração no banco e mensagem apresentada ao usuário
CQ-06	A string de conexão não está configurada no ambiente	A invocação falha com mensagem explícita de configuração ausente e registro na telemetria	Código 500 com motivo identificável, sem exposição de credencial
CQ-07	A equipe decide publicar o frontend em outro provedor de hospedagem estática	O mesmo artefato é publicado sem alteração de código	Apenas variáveis de ambiente de build são ajustadas
CQ-08	O gestor emite a proposta de um orçamento aprovado	O documento é paginado em A4 com identificação, folha de processo e demonstrativo de preço	Saída pronta para envio, e exportação estruturada equivalente disponível

11. Riscos e Dívidas Técnicas (Risks and Technical Debt)

Esta seção registra os **riscos arquiteturais** inerentes às decisões tomadas na seção 9. Cada linha descreve um *trade-off* assumido conscientemente, o sinal observável que indicaria a sua materialização e a estratégia de mitigação já definida. Não se trata de pendência de implementação: são propriedades conhecidas do desenho escolhido, monitoradas deliberadamente.

As fronteiras da arquitetura são sustentadas por mecanismos automatizados — diagramas mantidos como código-fonte e regerados por script, e regras de dependência verificadas por testes unitários de arquitetura (seção 8.7) — de modo que a erosão estrutural é detectada pela própria suíte, e não por inspeção manual.

Risco	Origem	Sinal de alerta	Mitigação prevista
Partida a frio na primeira invocação	Plano de consumo da API serverless (ADR-01)	Primeira consulta do dia com latência visivelmente maior no banner	Latência exposta ao usuário; se incomodar o fluxo, adotar plano com instância sempre pronta ou aquecimento programado
Crescimento do agregado de orçamento	Operações embutidas no documento (ADR-04)	Peças com folha de processo muito longa aproximando o documento do limite do banco	Monitorar o tamanho médio do documento; se necessário, mover o histórico de operações para coleção própria
Volume da listagem completa	Consulta que devolve todos os orçamentos para compor indicadores	Tempo de resposta da consulta crescendo com a base	Introduzir paginação e filtro no servidor, além de indicadores pré-agregados, mantendo o contrato atual como caso particular
Dependência de provedor de nuvem	Uso do modelo de funções e do banco gerenciado	Mudança de política ou de custo do provedor	Domínio e casos de uso não conhecem o provedor; a troca se concentra nos adaptadores de borda
Motor de cálculo executado no cliente	Decisão de manter a precificação no navegador para resposta imediata (ADR-02)	Preço exibido em tela divergindo de um recálculo feito fora da aplicação	Domínio isolado como módulo puro, sem dependência de framework, e coberto por testes de cálculo que fixam cada regra de rateio e de margem
Políticas transversais concentradas na própria API	Exposição direta do Function App, com o prefixo `/api` como contrato de borda (ADR-07)	Política transversal nova que exigisse alteração repetida em todas as funções	O prefixo estável permite inserir um gateway à frente do serviço sem alterar o cliente nem o contrato já publicado
Importação de planilha sensível a variações de formato	Diversidade de modelos de planilha do cliente (RF01, RF07)	Arquivos com colunas fora do padrão esperado	Validação de integridade antes da gravação, com relato das inconsistências ao usuário antes de confirmar

12. Glossário (Glossary)

Termos de domínio e de arquitetura usados neste documento, na acepção adotada pelo projeto.

Termo	Definição
Agregado	Conjunto de dados tratado como uma unidade de consistência. Aqui, o orçamento com suas operações e seu desenho técnico.
API Gateway	Componente de borda que centraliza autenticação, limitação de uso, versionamento de contrato e observabilidade das APIs. Não adotado neste escopo (ADR-07): o prefixo `/api` do Function App cumpre o papel de contrato de borda estável.
Arquitetura orientada a eventos (EDA)	Estilo em que componentes reagem de forma assíncrona a fatos de negócio publicados, em vez de se chamarem diretamente. Não adotado no fluxo principal (ADR-08), que é interativo e exige consistência imediata em tela.
Backend for Frontend (BFF)	Serviço intermediário dedicado a um tipo de cliente, que agrega e adapta respostas de vários serviços ao formato que aquela interface consome. Não adotado neste escopo (ADR-06): há um único consumidor e nenhuma agregação de fontes a fazer.
Banco por serviço (Database per Service)	Padrão em que cada serviço é proprietário exclusivo do seu banco, sem acesso compartilhado. Não adotado neste escopo (ADR-06), que mantém um agregado transacional único.
Centro de custo	Unidade contábil que agrupa recursos produtivos, identificada por código (por exemplo `CC-310`, usinagem pesada CNC) e associada a uma taxa horária média.
Clean Architecture	Organização em camadas na qual as dependências apontam para o domínio, mantendo regra de negócio independente de framework, interface e infraestrutura.
Composition root	Único ponto da aplicação onde os adaptadores concretos são instanciados e injetados nos casos de uso.
C4 Model	Notação de arquitetura em quatro níveis de abstração: contexto, contêineres, componentes e código.
Fatia vertical (Vertical Slice)	Agrupamento de código por funcionalidade, contendo todas as camadas necessárias para entregá-la.
Flag Zerar Custo	Marcação por operação que anula o custo efetivo daquela etapa no preço final, preservando seus tempos de setup e de ciclo.
Folha de processo	Sequência ordenada de operações de fabricação de uma peça, com centro de trabalho, tempos e custos.
Integrador_Reports	Módulo responsável pela proposta comercial consolidada e pela exportação estruturada do orçamento.
Lote padrão	Quantidade de peças considerada no rateio do tempo de preparação das máquinas.
Microfrontend	Estilo em que partes independentes de uma mesma interface são desenvolvidas e publicadas por equipes distintas. Não adotado neste escopo (ADR-05): uma equipe, um domínio central e artefato de publicação único.
Microsserviços	Estilo em que o sistema é decomposto em serviços pequenos, independentes em publicação e escala, cada um com a sua fronteira de dados. Não adotado neste escopo (ADR-06), que mantém um serviço coeso de orçamentação com a função por operação como granularidade de implantação.
Partida a frio	Latência adicional da primeira invocação de uma função serverless após período de inatividade.

Termo	Definição
Porta e adaptador (Ports & Adapters)	Contrato definido pela aplicação (porta) e implementado por um componente de borda (adaptador), que pode ser substituído sem afetar o domínio.
Rugosidade Ra	Média aritmética dos desvios do perfil de uma superfície usinada, em micrometros.
Serverless (FaaS)	Modelo de execução no qual a plataforma instancia a função sob demanda e cobra por invocação, sem servidor administrado pela equipe.
SPA (Single Page Application)	Aplicação web que carrega uma vez e atualiza a interface no próprio navegador, sem recarregar a página.
Setup	Tempo de preparação da máquina antes da produção do lote, amortizado no custo unitário da peça.
Teste unitário de arquitetura	Teste automatizado que verifica propriedades estruturais do código, como regras de dependência entre camadas e fatias.
Tolerância ISO 2768-mK	Classe de tolerâncias gerais de fabricação aplicada quando o desenho não especifica tolerância individual.